مجموعه‌مقالات عامل‌ها (agent) · می ۲۰۲۶

چرخهٔ جدید تولید نرم‌افزار با کدنویسی شهودی

از پرامپت‌های (prompt) نامنظم
تا مهندسی عامل‌محور (Agentic engineering)

نویسندگان: آدی عثمانی، شوبهام سابو،
و سوکراتیس کارتاکیس

مشارکت محتوا: الیا سکی، جولیا وایزینگر، آنانت ناوالگاریا · گردآوری و ویرایش: آنانت ناوالگاریا · طراحی: مایکل لنینگ

فهرست مطالب

1. مقدمه
    1. چرا این مقاله، چرا حالا
    2. این مقاله برای چه کسانی است
2. گذار از نحو به قصد
    1. عامل‌های هوش مصنوعی: یادآوری کوتاه
    2. کدنویسی شهودی چیست
    3. طیف: از کدنویسی شهودی تا مهندسی عامل‌محور
    4. مهندسی زمینه: مهارت واقعی
3. چرخهٔ جدید تولید نرم‌افزار
    1. چرخهٔ سنتی زیر فشار
    2. هوش مصنوعی هر مرحله را چگونه دگرگون می‌کند
    3. مدل کارخانه
4. مهندسی مهار: آنچه دور مدل ساخته می‌شود
    1. درون مهار چیست
    2. مهار در چرخهٔ تولید
5. نقش در حال تحول توسعه‌دهنده
    1. رهبر ارکستر

2. هماهنگ‌کننده
3. مسئلهٔ ۸۰ درصد
6. عامل‌های کدنویسی در عمل
1. جای عامل‌ها در روز کاری توسعه‌دهنده
2. کدنویسی شهودیِ عامل‌های آمادهٔ تولید
7. هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی
1. بدهی پنهان کدنویسی شهودی
2. سرمایه‌گذاری مهندسی عامل‌محور
3. مهندسی زمینه به‌مثابه اهرم مالی
8. از کجا شروع کنیم
9. نتیجه‌گیری: قصد به‌مثابه رابط تازه
10. پی‌نوشت‌ها

بزرگ‌ترین تحول در مهندسی نرم‌افزار، یک زبان، چارچوب یا سرویس ابری تازه نیست؛ گذار از نوشتن کد به بیان قصد (`intent`) است، و سپردن این کار به سامانه‌های هوشمند که آن قصد را به نرم‌افزاری کارآمد تبدیل کنند.

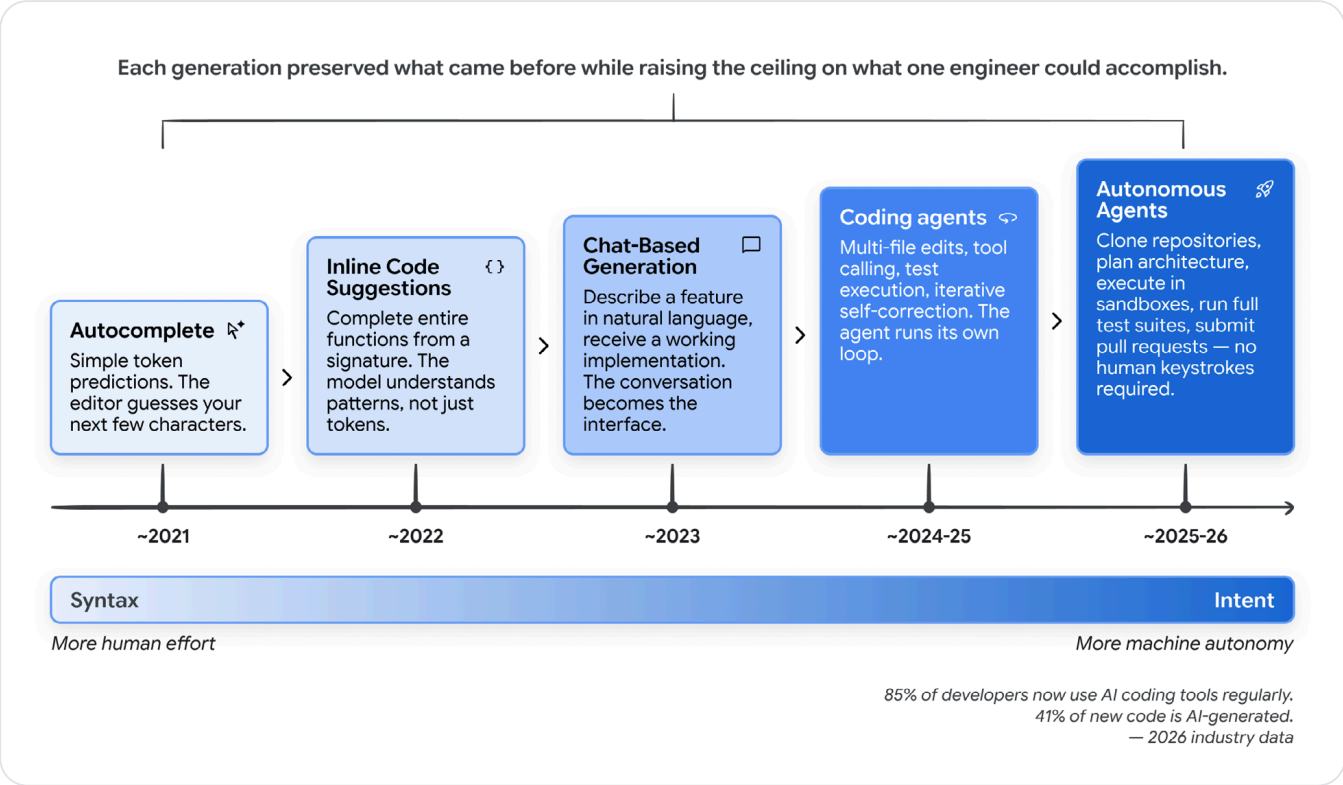
۱. مقدمه

در بیشتر تاریخ رایانش، برنامه‌نویسی در اصل نوعی ترجمه بوده است: مسئله را به زبان انسانی بفهم، راه‌حلی انتزاعی برایش طراحی کن، و بعد آن را با نحوی بازنویسی کن که ماشین بتواند اجرایش کند. هر کدام از این گام‌ها دردسر و کندی خودش را دارد، و این موانع حالا دارند از میان می‌روند. مهندسی نرم‌افزار امروز بزرگ‌ترین تحول خود را از زمان پیدایش زبان‌های برنامه‌نویسی سطح‌بالا تجربه می‌کند.

دهه‌ها، اصلی‌ترین راه ارتباط توسعه‌دهنده با ماشین **نحو** بوده است: آکولاد، نقطه‌ویرگول، تعیین نوع متغیرها، و دستور زبان دقیق زبان‌های برنامه‌نویسی. آن دوران دارد به پایان می‌رسد.

پارادایمی تازه از راه رسیده که در آن توسعه‌دهندگان بیان می‌کنند **چه** می‌خواهند بسازند، نه اینکه **چگونه** بسازندش. پیاده‌سازی با ماشین است؛ قصد، معماری و قضاوت با انسان. این آینده‌ای دور نیست؛ همین حالا کار روزمرهٔ شمار رو به رشدی از توسعه‌دهندگان حرفه‌ای است. تا اوایل ۲۰۲۶، **۸۵ درصد** از توسعه‌دهندگان حرفه‌ای به‌طور منظم از عامل‌های کدنویسی استفاده می‌کنند و **۵۱ درصد** هر روز؛ و برآورد می‌شود **۴۱ درصد** از همهٔ کدهای تازه را هوش مصنوعی تولید می‌کند.

این تحول یک‌شبه رخ نداد. با تکمیل خودکار شروع شد، یعنی پیش‌بینی سادهٔ توکن (token) بعدی در ویرایشگر. سپس پیشنهادهای درون‌خطی کد آمدند که می‌توانستند یک تابع کامل را بنویسند. بعد رابط‌های گفت‌وگومحور اجازه دادند توسعه‌دهندگان قابلیت‌ها را به زبان طبیعی توصیف کنند و پیاده‌سازی کارآمدی دریافت کنند. حالا عامل‌های کاملاً خودگردان (autonomous) می‌توانند ریپوها را کلون کنند، تغییرهایی را که چند فایل را درگیر می‌کنند برنامه‌ریزی کنند، آن‌ها را در محیط‌های سندباکسی اعمال کنند، تست‌ها (test) را اجرا کنند و پول‌ریکوئست (pull request) ثبت کنند — همه بدون آنکه انسانی حتی یک خط کد تایپ کند.



شکل ۱: از تکمیل خودکار تا خودگردانی.

پیامدهای این پارادایم جدید برای چرخهٔ تولید نرم‌افزار عمیق‌اند. هر مرحله — از گردآوری نیازمندی‌ها تا دیپلوی (deploy) و نگه‌داری — دارد با توانمندی‌های هوش مصنوعی بازتعریف می‌شود. اما این دگرگونی یکنواخت یا ساده نیست، بلکه طیفی دارد. یک سرِ آن «کدنویسی شهودیِ» سرِدستی است: توسعه‌دهنده پرامپتی (prompt) به هوش مصنوعی می‌دهد و هرچه برگردد می‌پذیرد. سر دیگرش «مهندسی عامل‌محورِ» ساختارمند است. در این سر، هوش مصنوعی موتور قدرتمند پیاده‌سازی است، اما درون چارچوبی کار می‌کند که با دقت طراحی شده است: قیدها، تست‌ها و حلقه‌های (loop) بازخورد. نظارت بر معماری، درستی و کیفیت هم همچنان با انسان‌هاست.

چرا این تمایز اهمیت دارد

اگر به مدیر ارشد فناوری بگویید تیم شما سامانهٔ پردازش پرداخت را «شهودی» می‌نویسد، زنگ خطر به صدا درمی‌آید — و باید هم درآید. اما اگر به همان مدیر بگویید تیم شما **مهندسی عامل‌محور** می‌کند، گفت‌وگو از بنیاد فرق می‌کند: هوش مصنوعی کد را می‌نویسد، اما در چارچوبی که انسان‌ها تعیین کرده‌اند، و پوشش (coverage) تست درستی کار را تضمین می‌کند.

چرا این مقاله، چرا حالا

ابزارها، توانمندی‌ها و پارادایم‌های تازه هفتگی پدیدار می‌شوند. تیم‌های مهندسی به چارچوبی برای فهم این چشم‌انداز نیاز دارند — نه عکسی لحظه‌ای که ظرف چند ماه کهنه شود، بلکه مجموعه‌ای از اصول و مدل‌های ذهنی که با تکامل ابزارهای مشخص همچنان سودمند بمانند.

این مقاله برای چه کسانی است

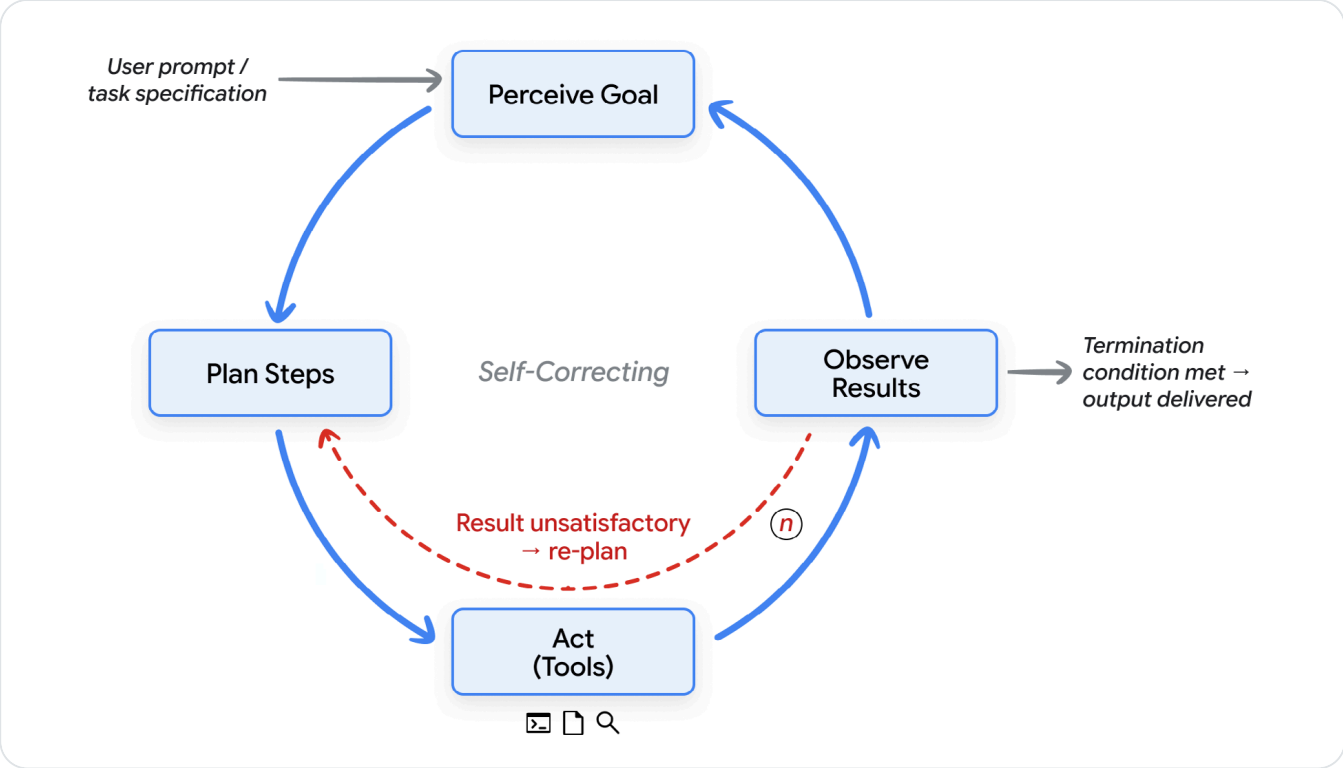
این مقاله برای مهندسان نرم‌افزار، مدیران مهندسی، معماران و رهبران فنی است که می‌خواهند بفهمند هوش مصنوعی چگونه چرخهٔ تولید نرم‌افزار را بازتعریف می‌کند، و این توانمندی‌های تازه را بدون فداکردن انضباطی که نرم‌افزار تولیدی می‌طلبد به کار بگیرند. آشنایی با رویه‌های مدرن توسعهٔ نرم‌افزار را فرض گرفته‌ایم، اما آشنایی با جزئیات هوش مصنوعی یا یادگیری ماشین را نه.

۲. گذار از نحو به قصد

پیش از آنکه جلوتر برویم، به تصویری مشترک از اینکه عامل چیست و کدنویسی شهودی (Vibe coding) واقعاً چه معنایی دارد نیاز داریم. در هر دو اصطلاح آن‌قدر معنا انباشته شده که باید با دقت از هم بازشان کرد.

عامل‌های هوش مصنوعی: یادآوری کوتاه

عامل هوش مصنوعی سامانه‌ای نرم‌افزاری است که هدفی را ادراک می‌کند، گام‌هایی برای رسیدن به آن برنامه‌ریزی می‌کند، از راه ابزارها کنش انجام می‌دهد، نتایج را مشاهده می‌کند، و آن‌قدر تکرار می‌کند تا هدف برآورده شود یا به شرط توقفی برسد. جایی که یک ربات گفت‌وگو پاسخی تولید می‌کند و منتظر پرامپت بعدی می‌ماند، عامل **حلقهٔ خودش** را اجرا می‌کند. شما در بالا هدفی به او می‌دهید، و او در هر گام تصمیم می‌گیرد بعد چه کند.



شکل ۲: حلقهٔ عامل — ادراک، برنامه، کنش، مشاهده، تکرار.

هر عامل، هرقدر ساده یا پیچیده، از پنج بخش ساخته شده است:

- **مدل** موتور استدلال است. زمینهٔ کنونی را می‌خواند، تصمیم می‌گیرد بعد چه باید رخ دهد، و تفکر `(thinking)` بعدی، فراخوان ابزار `(tool call)` بعدی یا پیام بعدی را تولید می‌کند.
- **ابزارها** مدل را به جهان وصل می‌کنند. شامل رابط‌هایی که عامل می‌تواند فراخوانی کند، کدی که می‌تواند اجرا کند، پایگاه‌های داده‌ای که می‌تواند پرس‌وجو کند، و عامل‌های دیگری که می‌تواند کاری را به آن‌ها واگذار کند.
- **حافظه** `(memory)` وضعیت است. به عامل اجازه می‌دهد تعامل‌های گذشته را به یاد بیاورد، قواعد ویژهٔ پروژه را بازیابی `(retrieval)` کند، و زمینه `(context)` را میان نشست‌ها `(session)` نگه دارد تا هرگز از صفحهٔ سفید شروع نکند.
- **ارکستراسیون** `(orchestration)` کدی است که حلقه را اجرا می‌کند. زمینه را برای هر فراخوان مدل سرِهم می‌کند، فراخوان‌های ابزار `(tool)` را اجرا می‌کند، نتایجشان را می‌گیرد، و تصمیم می‌گیرد ادامه بدهد یا نه.
- **دیپلوی** همان چیزی است که نمونهٔ اولیه را به سرویس بدل می‌کند: میزبانی، هویت، رصدپذیری، و زیرساخت تولیدی که عامل روی آن اجرا می‌شود.

این بخش‌ها در حلقه‌ای پیوسته با هم کار می‌کنند: مأموریت را بگیر، صحنه را برپا کن، خوب فکر کن، کنش انجام بده، مشاهده کن و تکرار کن. **حلقه، قلب تپندهٔ هر عامل است.** هر چیز دیگری در این مقاله، و هر چیزی در باقی دوره، گونه‌ای از همین حلقه است.

کدنویسی شهودی چیست

در فوریهٔ ۲۰۲۵، آندری کارپثی توصیفی از شیوهٔ تازه‌ای از برنامه‌نویسی منتشر کرد که در سراسر جامعهٔ مهندسی نرم‌افزار مورد استقبال و استفاده قرار گرفت. او رویکردی را وصف کرد که در آن کاملاً تسلیم «حس‌وحال» می‌شوید، رشد شتابان توانایی‌های هوش مصنوعی را می‌پذیرید و فراموش می‌کنید که اصلاً کدی وجود دارد. در این حالت، توسعه‌دهنده خواسته‌اش را به زبان طبیعی توصیف می‌کند، خروجی هوش مصنوعی را می‌پذیرد، و وقتی چیزی خراب شد، پیام خطا را دوباره در پرامپت می‌گذارد و از هوش مصنوعی می‌خواهد درستش کند.

این اصطلاح فراگیر شد چون موضوعی واقعی را پوشش داده بود: بسیاری از توسعه‌دهندگان از قبل این‌طور کار می‌کردند اما اصطلاحی برای توصیفش نداشتند. ظرف چند ماه، «کدنویسی شهودی» به توصیف‌گر رایج هر گردش‌کار (workflow) توسعهٔ مبتنی بر هوش مصنوعی بدل شد، و همین سردرگمی آفرید. آیا مهندس ارشدی که از دستیار هوش مصنوعی برای پیاده‌سازی قابلیتی که به‌خوبی مشخص شده استفاده می‌کند «کدنویسی شهودی» می‌کند؟ آیا تیمی که از عامل‌ها برای اجرای معماری‌ای بادقت‌برنامه‌ریزی‌شده بهره می‌گیرد، «کدنویسی شهودی» می‌کند؟ اصطلاح آن‌قدر گسترده به کار رفت که شروع به ازدست‌دادن معنا کرد.

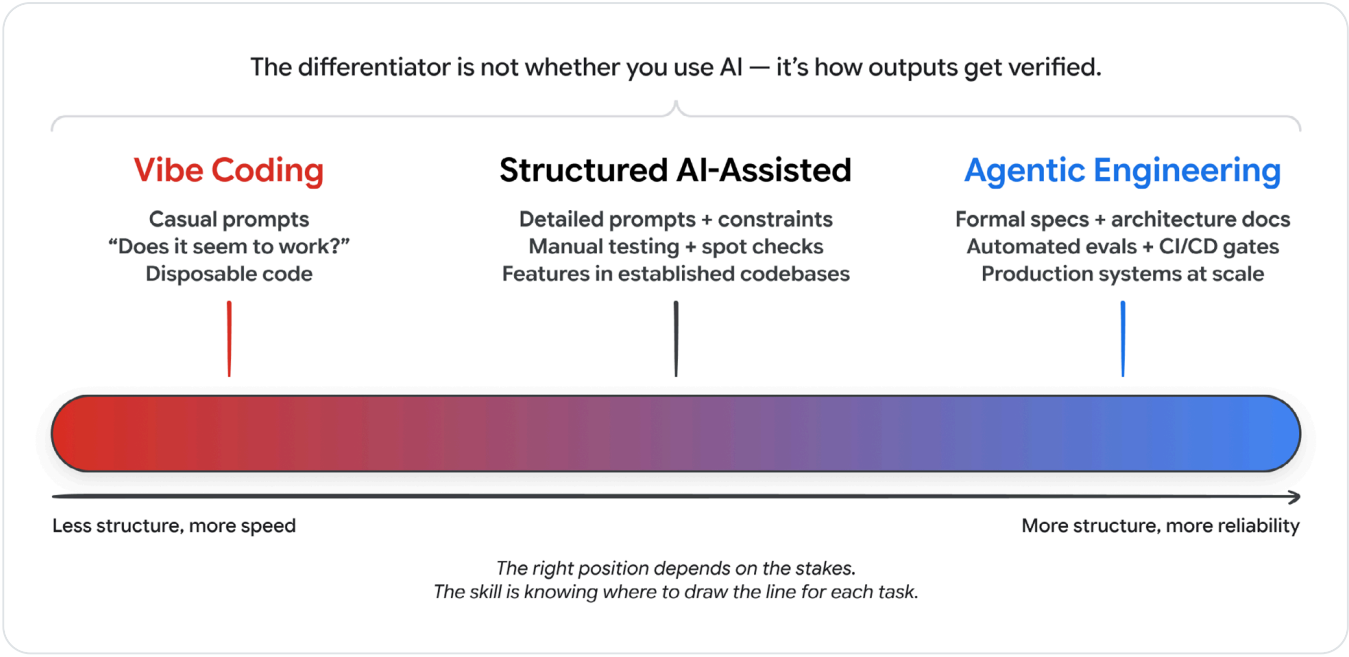
تا اوایل ۲۰۲۶، خود کارپثی پذیرفت که تعریف اولیه‌اش بیش از حد محدود بوده است و اصطلاح **«مهندسی عامل‌محور»** را برای سرِ ساختارمندترِ این طیف معرفی کرد.

طیف: از کدنویسی شهودی تا مهندسی عامل‌محور

به‌جای اینکه کدنویسی شهودی و مهندسی عامل‌محور را یک دو انتخابی بدانیم، سودمندتر است آن‌ها را نقاط انتهایی یک **طیف** در نظر بگیریم. تمایزدهندهٔ کلیدی این نیست که آیا از هوش مصنوعی استفاده می‌کنید یا نه. این است که **چه میزان ساختار، راستی‌آزمایی و قضاوت انسانی گرد خروجی هوش مصنوعی را گرفته است.**

بُعد	کدنویسی شهودی	کدنویسی ساختاریافته با کمک هوش مصنوعی	مهندسی عامل‌محور
بیان قصد	پرامپت‌های سرِدستی به زبان طبیعی	پرامپت‌های مفصل با مثال و قید (constraint)	مشخصات (spec) رسمی، اسناد معماری، فایل‌های حافظه
راستی‌آزمایی	«به‌نظر می‌رسد کار می‌کند؟»	تست دستی، وارسی نقطه‌ای	مجموعه‌تست‌های خودکار، دروازه‌های (gate) سی‌آی، داوران مدل‌زبانی
درک کدبیس	حداقلی؛ توسعه‌دهنده ممکن است کد تولیدشده را نخواند	بازبینی گزینشی مسیرهای بحرانی	بازبینی جامع معماری؛ هوش مصنوعی جزئیات پیاده‌سازی را اداره می‌کند
مدیریت خطا	کپی‌کردن پیام خطا و برگرداندنش به هوش مصنوعی	توسعه‌دهنده علت ریشه‌ای را تشخیص می‌دهد، هوش مصنوعی اصلاح را پیاده می‌کند	عامل‌ها درون مرزهای تعریف‌شده خودتشخیصی می‌کنند؛ انسان‌ها مسائل معماری را اداره می‌کنند
دامنهٔ مناسب	نمونه‌های اولیه، اسکریپت‌ها، پروژه‌های شخصی، هکاتون‌ها	افزودن قابلیت به کدبیس‌های موجود	سامانه‌های تولیدی، توسعه در مقیاس تیم
نمای مخاطره	بالا؛ برای کد دورریختنی (Disposable code) پذیرفتنی	متوسط؛ قضاوت انسانی در نقاط وارسی کلیدی	پایین؛ راستی‌آزمایی نظام‌مند در هر مرحله

جدول ۱ — طیف از کدنویسی شهودی تا مهندسی عامل‌محور.



شکل ۳: طیف کدنویسی شهودی تا مهندسی عامل‌محور.

نکتهٔ کاربردی

جایگاه درست روی این طیف به میزان خطر بستگی دارد. یک نمونهٔ اولیهٔ آخر هفته می‌تواند کدنویسی شهودی خالص باشد. اما رابطی تولیدی که تراکنش‌های مالی را اداره می‌کند مهندسی عامل‌محور می‌طلبد. بیشتر کار واقعی جایی در میانه می‌افتد، و مهارت در این است که بدانید برای هر کار خط را کجا بکشید.

بزرگ‌ترین تمایزدهنده: راستی‌آزمایی

در کدنویسی شهودی، راستی‌آزمایی اختیاری است؛ توسعه‌دهنده کد را اجرا می‌کند و می‌بیند درست به‌نظر می‌رسد یا نه. در مهندسی عامل‌محور، دو سازوکار با هم کار می‌کنند. **تست‌ها** بخش‌های قطعی `(deterministic)` سامانه را راستی‌آزمایی می‌کنند: تابعی با این ورودی، آن خروجی را می‌دهد. **ارزیابی‌ها** `(evaluation)` بخش‌های غیرقطعی `(not deterministic)` را: آیا عامل مسیر `(trajectory)` درستی از گام‌ها را طی کرد، ابزارهای درست را برگزید، و پاسخ نهایی‌ای تولید کرد که حد کیفیت را برآورده کند.

تست‌ها با کد وارسی می‌شوند، اما ارزیابی‌ها به مجموعه‌داده‌های برچسب‌خورده، سنجه‌های `(rubric)` نمره‌دهی و داوران مدل‌زبانی نیاز دارند. **بدون هر دو، رویه همیشه کدنویسی شهودی است، صرف‌نظر از اینکه پرامپت‌ها چقدر پیچیده باشند.**

مهندسی زمینه: مهارت واقعی

با بلوغ این حوزه، بینشی کلیدی پدیدار شده است: کیفیت کد تولیدشده با هوش مصنوعی کمتر به زیرکی پرامپت‌های شما بستگی دارد و بیشتر به **کیفیت زمینه‌ای که فراهم می‌کنید** وابسته است. این دریافت به مفهوم **مهندسی زمینه** (Context engineering) انجامیده است: رویهٔ فراهم‌کردن اطلاعات غنی و ساختارمند دربارهٔ کدبیس، معماری، قراردادها و قصد شما برای عامل‌های هوش مصنوعی.

توسعه‌دهندگان باید شش نوع اصلی زمینه را در نظر بگیرند:

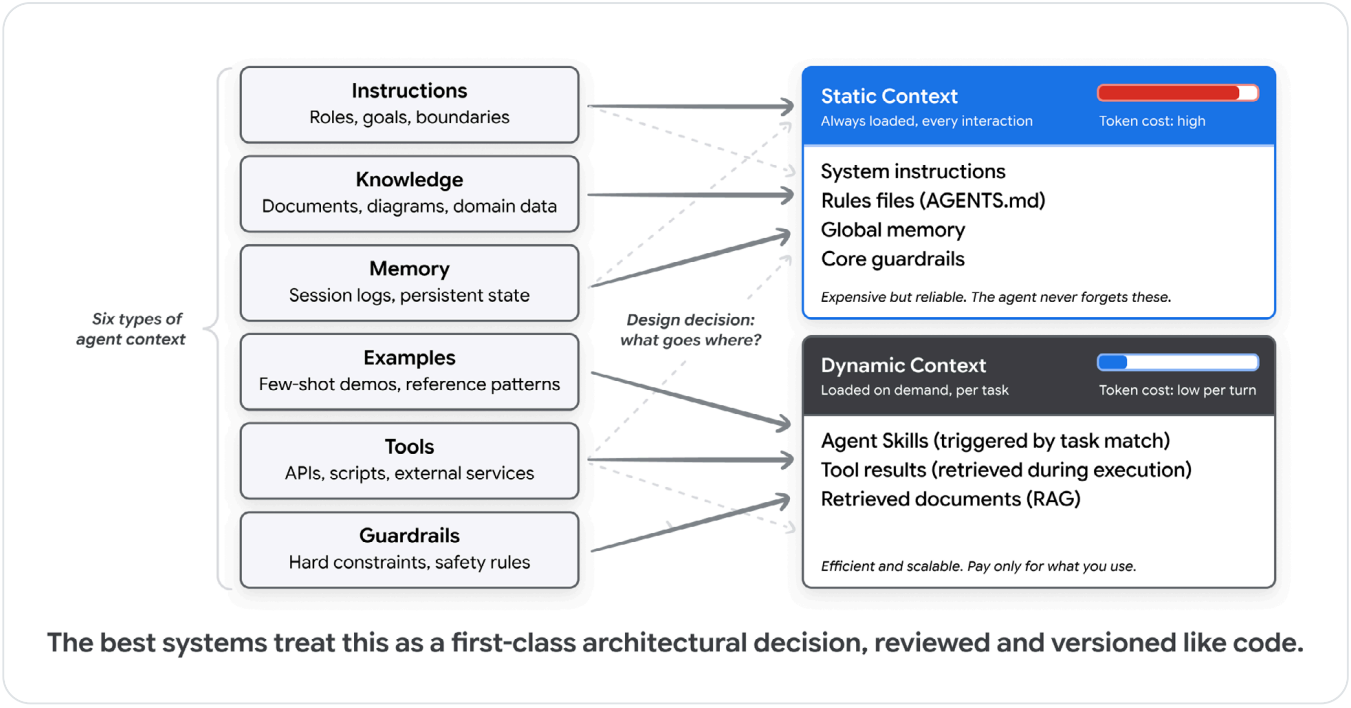
- **دستورها** (Instructions): نقش هسته‌ای عامل، هدف‌ها و مرزهای عملیاتی‌اش.
- **دانش** (Knowledge): اسناد بازیابی‌شده، نمودارهای معماری، و دادهٔ ویژهٔ حوزه.
- **حافظه** (Memory): گزارش‌های کوتاه‌مدت نشست (چه چیزی همین حالا رخ داد) و وضعیت ماندگار بلندمدت (پروژه چیست).
- **مثال‌ها** (Examples): نمایش‌های رفتاری چندنمونه‌ای و الگوهای مرجع کدبیس.
- **ابزارها** (Tools): تعریف دقیق رابط‌ها، اسکریپت‌ها و سرویس‌های بیرونی که عامل می‌تواند فرا بخواند.
- **گاردریل‌ها** (guardrail): قیدهای سخت، قواعد قالب‌بندی، و اعتبارسنجی‌های ایمنی.

در تولید کد با هوش مصنوعی، مهندسی زمینه یعنی متوازن‌کردن بادقتِ اینکه عامل کدام‌یک از این شش عنصر را از پیش در اختیار دارد و کدام را می‌تواند هنگام نیاز بازیابی کند. این جدایی حیاتی میان **زمینهٔ ایستا** (Static) و **زمینهٔ پویا** (dynamic) را می‌سازد.

زمینهٔ ایستا همیشه بارگذاری شده است: سیستم پرامپت‌ها، فایل‌های قاعده (`AGENTS.md` ، `CLAUDE.md` ، `GEMINI.md`)، حافظهٔ سراسری، و تعریف شخصیت. این تعیین می‌کند عامل کیست و چطور رفتار می‌کند. زمینهٔ ایستا گران است، چون هر توکن آن در هر تعامل حاضر است، صرف‌نظر از اینکه مرتبط باشد یا نه.

زمینهٔ پویا هنگام نیاز بارگذاری می‌شود: دستورهای مهارت (skill) که درصورت تطبیق با کار، فعال می‌شوند، نتایج ابزار که حین اجرا بازیابی می‌شوند، اسنادی که از خطوط لولهٔ RAG دریافت می‌شوند، و بخش اخیر تاریخچهٔ نشست. زمینهٔ پویا کارآمد است، چون عامل هزینهٔ توکن را فقط وقتی می‌پردازد که اطلاعات لازم باشد.

> **این مرز یک تصمیم معماری درجه‌یک است**
>
> اینکه چه چیزی به زمینهٔ ایستا تعلق دارد و چه چیزی به زمینهٔ پویا، یک موازنهٔ مهندسی واقعی است. زمینهٔ ایستای بیش‌ازحد زیاد، توکن هدر می‌دهد و سیگنال را رقیق می‌کند. زمینهٔ ایستای بیش‌ازحد کم یعنی عامل قواعد حیاتی را فراموش می‌کند. سامانه‌های خوب طراحی شده، با این مرز مانند هر پیکربندی دیگری رفتار می‌کنند: بازبینی‌شده و نسخه‌بندی‌شده.



شکل ۴: مهندسی زمینه — ایستا در برابر پویا.

نیرومندترین الگو برای مدیریت زمینهٔ پویا، **مهارت‌های عامل** است: بسته‌های ساختارمند و قابل‌حمل دانش رویه‌ای (`procedural knowledge`) که عامل فقط وقتی آن‌ها را بارگذاری می‌کند که کار اقتضا کند.

به‌جای جاسازی هر تکه دانش تخصصی در سیستم پرامپت عامل، مهارت‌ها به عامل اجازه می‌دهند یک عامل عمومی سبک بماند که به‌اقتضا و از راه **افشای تدریجی** (`Progressive disclosure`) در نقش‌های تخصصی فرو می‌رود. عامل در آغاز فقط فرادادهٔ سبک را می‌بیند، وقتی کاری تطبیق پیدا کرد دستورهای کامل را بارگذاری می‌کند، و مطالب مرجع تفصیلی را فقط وقتی واقعاً لازم شود می‌خواند. نتیجه این است که عامل می‌تواند ده‌ها توانمندی تخصصی حمل کند، در حالی که هزینهٔ توکن را فقط برای همانی می‌پردازد که فعالانه به کار می‌برد.

مهارت‌های عامل در عامل‌های کدنویسی و سکوهای سازمانی به‌سرعت پذیرفته شده‌اند، چون چهار مسئله را حل می‌کنند:

- پوسیدگی زمینه (`Context rot`) بر اثر پرامپت‌هایی که بیش از حد در آن‌ها مطلب گنجانده شده
- نبود حافظهٔ رویه‌ای (`Procedural memory`) برای مدل‌های زبانی بزرگ
- سربار عملیاتی معماری‌های چندعاملی
- نیاز به قابلیت حمل میان ابزارها و ارائه‌دهندگان

> مدل‌ها بیش از پرامپت‌های ماهرانه، به زمینهٔ کافی نیاز دارند؛ همان زمینه‌ای که یک توسعه‌دهندهٔ ماهر هم برای انجام کار خوب لازم دارد. پرسش این نیست که «چطور هوش مصنوعی را فریب بدهم تا کد خوب بنویسد؟» بلکه این است که «یک عضو تازهٔ تیم چه چیزی باید بداند تا مشارکت مؤثر داشته باشد، و چطور آن دانش را به شکلی در بیاورم که هوش مصنوعی بتواند به کار ببرد؟»

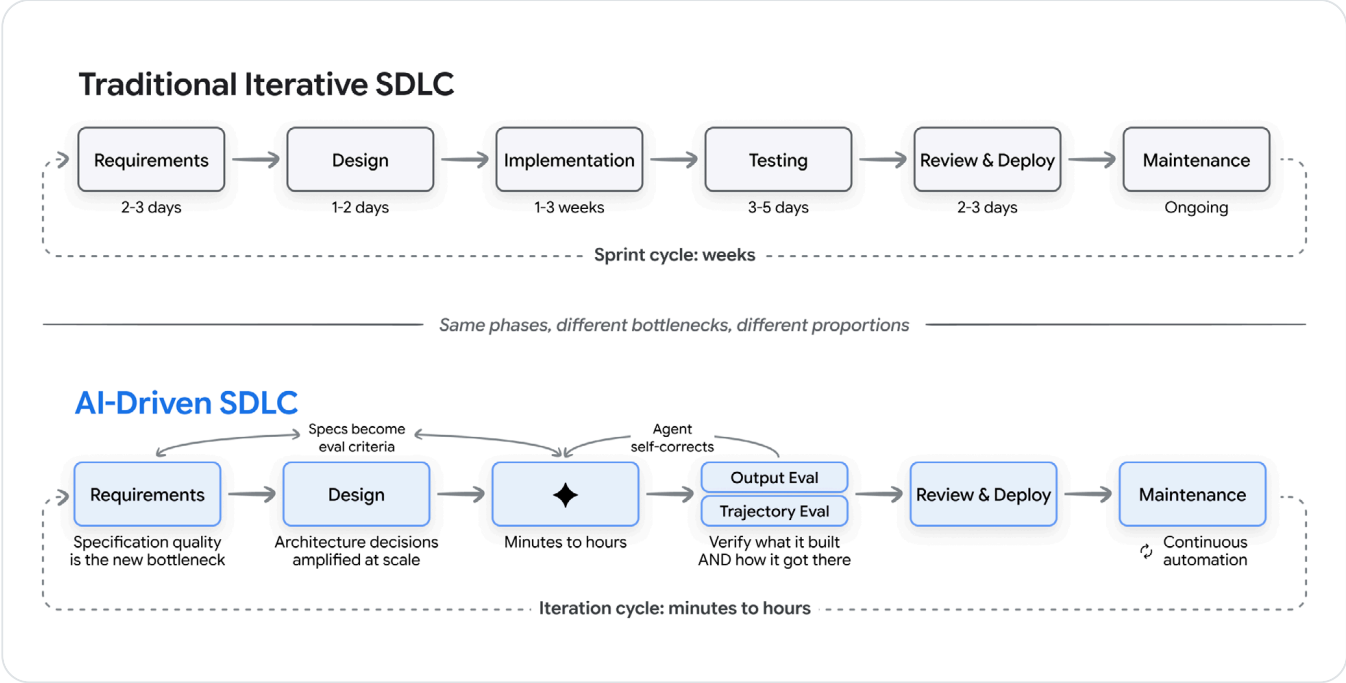
مهندسی زمینه پل میان کدنویسی شهودی و مهندسی عامل‌محور است. با جابه‌جایی تمرکز از نوشتن نحو به مهندسی زمینه، گلوگاه‌های ساخت نرم‌افزار بنیاداً تغییر می‌کنند. ما دیگر منتظر دستان انسانی برای تایپ کد تکراری نیستیم؛ منتظر ذهن‌های انسانی هستیم تا مرزها را تعریف کنند.

۳. چرخهٔ جدید تولید نرم‌افزار

چرخهٔ سنتی زیر فشار

چرخهٔ تولید نرم‌افزار تاکنون یک دگرگونی بزرگ را از سر گذرانده است. در دو دههٔ گذشته، بیشتر سازمان‌ها از فرایندهای متوالی آبشاری به مدل‌های تکرارشونده کوچ کردند: دوره‌های چابک، سی‌آی، خطوط `DevOps`، و چرخه‌های انتشار سریع. آن دگرگونی حلقه‌های بازخورد را کوتاه کرد، تست را به توسعه نزدیک‌تر ساخت، و دیپلوی را از رویدادی فصلی به فرایندی پیوسته بدل کرد.

هوش مصنوعی این چرخه را به‌شدت فشرده می‌کند، اما **نابرابر**: پیاده‌سازی‌ای که زمانی هفته‌ها طول می‌کشید حالا می‌تواند در یک ساعت انجام شود، در حالی که نیازمندی‌ها، معماری و راستی‌آزمایی همچنان با سرعت انسانی پیش می‌روند و شتاب‌گرفتن آن‌ها به این آسانی ممکن نیست. نتیجه نسخهٔ سریع‌تری از چرخهٔ قدیمی نیست. **گردش‌کاری متفاوت** است، که در آن مرزهای میان مراحل محو می‌شوند، چرخه‌های تکرار از هفته به دقیقه کوتاه می‌شوند، و نقش توسعه‌دهنده از پیاده‌ساز اصلی به طراح سامانه و داور `(judge)` کیفیت جابه‌جا می‌شود.



شکل ۵: چرخهٔ سنتی تولید نرم‌افزار در برابر چرخهٔ هوش‌مصنوعی‌محور.

یادداشتی دربارهٔ سرعت تغییر

تصویر مرحله‌به‌مرحله‌ای که در ادامه می‌آید، وضعیت چرخهٔ هوش‌مصنوعی‌محور را تا میانهٔ ۲۰۲۶ بازتاب می‌دهد و به‌سرعت در حال جابه‌جایی است. نشانه‌های اولیه می‌گویند فشردگی فراتر از پیاده‌سازی خواهد رفت: تیم‌ها همین حالا هم گردش‌کارهایی را می‌آزمایند که در آن‌ها توسعه‌دهندگان مستقیم از مشخصات به بازبینی می‌روند و عامل‌ها پیاده‌سازی، تست و دیپلوی را در پس‌زمینه انجام می‌دهند. **آنچه ثابت می‌ماند، قضاوت انسانی، ذوق، و مهارت راستی‌آزمایی خروجی هوش مصنوعی است.**

هوش مصنوعی هر مرحله را چگونه دگرگون می‌کند

نیازمندی‌ها و برنامه‌ریزی

نیازمندی‌ها مرحله‌ای است که همیشه بیشترین فاصله را میان آنچه واقعاً خواسته شده و آنچه در عمل ساخته می‌شود داشته است. ترجمهٔ نیازهای کسب‌وکار به مشخصات فنی فرایندی دستی و خطاپذیر بوده که شکافی پایدار میان آنچه ذی‌نفعان می‌خواهند و آنچه مهندسان می‌سازند می‌آفریند.

ابزارهای مدرن می‌توانند مستقیم در پالایش نیازمندی‌ها مشارکت کنند: تولید داستان‌های کاربر از خلاصه‌های محصول، شناسایی حالت‌های مرزی که انسان‌ها از دست می‌دهند، تولید اسکیمای API‌ها از توصیف‌های زبان طبیعی، و ساخت نمونه‌های اولیهٔ تعاملی از اسناد مشخصات. **نیازمندی‌ها دیگر سندی نیست که میان تیم‌ها دست‌به‌دست شود؛ گفت‌وگویی میان انسان و هوش مصنوعی است که هم‌زمان مشخصات و پیاده‌سازی اولیه تولید می‌کند.**

طراحی و معماری

معماری سرسختانه‌ترین مرحلهٔ انسان‌محور چرخه باقی می‌ماند، و دلیل خوبی هم دارد. تصمیم‌های معماری بنیاداً دربارهٔ موازنه‌اند: سازگاری در برابر دسترس‌پذیری، پیچیدگی در برابر انعطاف، ساختن در برابر خریدن. این موازنه‌ها به زمینهٔ کسب‌وکار، قیدهای سازمانی و ملاحظات راهبردی بلندمدتی بستگی دارند که هوش مصنوعی نمی‌تواند کاملاً درکشان کند.

هوش مصنوعی در **پیاده‌کردن** تصمیم‌های معماری پس از گرفته‌شدنشان عالی است. با سند معماری روشن، عامل‌ها می‌توانند اسکلت کل برنامه‌ها را بسازند، الگوهای یکدست را در ماژول‌ها تولید کنند، و تضمین کنند کد تازه با قراردادهای موجود سازگار است.

پیاده‌سازی

عامل‌های کدنویسی مدرن می‌توانند کل قابلیت‌ها را از توصیف‌های زبان طبیعی تولید کنند، الگوریتم‌های پیچیده را پیاده کنند، و تغییرهایی در چند فایل بسازند که درست با هم کار می‌کنند. بهره‌وری واقعی است: پیمایش‌های صنعتی بهبود **۲۵ تا ۳۹ درصدی** گزارش می‌دهند.

> **اما واقعیت پیچیده‌تر از آن چیزی است که آمارهای پرسروصدا نشان می‌دهند**
>
> مطالعه‌ای از METR نشان داد توسعه‌دهندگان باتجربه‌ای که از دستیار هوش مصنوعی استفاده می‌کردند، روی برخی کارها در واقع **۱۹ درصد بیشتر** وقت گذاشتند — عمدتاً به‌خاطر زمانی که صرف راستی‌آزمایی، دیباگ (debug) و اصلاح خروجی هوش مصنوعی شد. **هوش مصنوعی کار پیاده‌سازی را حذف نمی‌کند، بلکه آن را از نوشتن به بازبینی، هدایت و راستی‌آزمایی دگرگون می‌کند.**

تست و تضمین کیفیت

برای آزمودن کدی که هوش مصنوعی تولید کرده، کافی نیست فقط نتیجهٔ کار را بسنجیم؛ باید ببینیم عامل **چگونه** به آن نتیجه رسیده است. ارزیابی خروجی، محصول نهایی را بررسی می‌کند: آیا کد کامپایل می‌شود و تست‌ها پاس می‌شوند؟ ارزیابی مسیر، توالی کامل فراخوان‌های ابزار و استدلال میانی را وارسی می‌کند. هر دو لازم‌اند، چون خروجی‌ای که ظاهری درست دارد اما گام‌های راستی‌آزمایی را جا انداخته، از خروجی‌ای با خطای آشکار خطرناک‌تر است.

مهم‌تر اینکه تست‌ها و ارزیابی‌ها به سازوکار اصلی انتقال قصد به عامل‌ها بدل می‌شوند: مجموعه‌ارزیابی‌ای که خوب نوشته شده باشد، به هوش مصنوعی می‌گوید «درست» یعنی چه، و راهی خودکار برای راستی‌آزمایی‌اش فراهم می‌کند. این رویه‌ها وقتی مؤثرترند که در یک **چرخهٔ پیوستهٔ بهبود کیفیت** به هم متصل شوند: ارزیابی در برابر مجموعه‌بنچمارک، تشخیص خطاها با خوشه‌بندی علت‌های ریشه‌ای، بهینه‌سازی پرامپت‌ها یا ابزارهایی که باعثشان شدند، راستی‌آزمایی اصلاح‌ها در برابر مجموعه‌تست پس‌رفت (regression suite)، و پایش ترافیک تولید برای یافتن حالت‌های شکست (failure modes) تازه. هر دور از این چرخه، بر دستاوردهای دور قبل افزوده می‌شود.

بازبینی کد و دیپلوی

خودِ فرایند بازبینی هم در حال تقویت است. هوش مصنوعی به‌عنوان بازبین نوبت اول عمل می‌کند و پیش از آنکه بازبین انسانی کد را ببیند، اشکال‌های بالقوه، نقض‌های سبکی، آسیب‌پذیری‌های امنیتی و مسائل کارایی را شناسایی می‌کند. این جایگزین بازبینی انسانی نمی‌شود — تصمیم‌های وابسته به زمینه دربارهٔ طراحی، نگه‌داشت‌پذیری و هم‌راستایی راهبردی همچنان قضاوت انسانی می‌طلبند — اما بار شناختی بازبینان را به‌شدت کاهش می‌دهد.

نگه‌داری و تکامل

شاید بزرگ‌ترین تغییری که کمتر به آن توجه شده، در نگه‌داری نرم‌افزار رخ داده باشد. کدبیس‌های قدیمی که روزگاری سر درآوردن از آن‌ها برای اعضای تازهٔ تیم تقریباً ناممکن بود، حالا با کمک هوش مصنوعی قابل پیمایش، فهم و اصلاح‌اند.

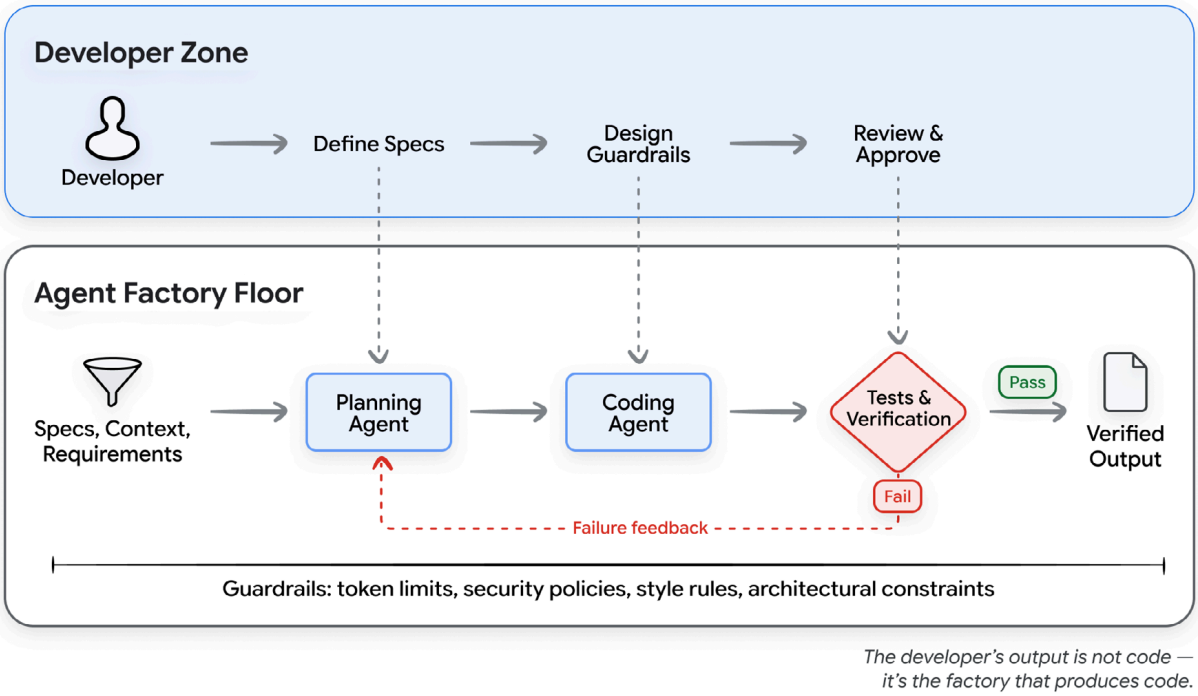
این تغییر اثر بزرگی بر بدهی فنی دارد. کدی که چون فقط نویسندگان اصلی‌اش از آن سر درمی‌آوردند، کسی جرئت دست‌زدن به آن را نداشت، حالا می‌تواند با اطمینان بازسازی، نوسازی و گسترش داده شود. عامل‌ها می‌توانند کدبیس‌ها را به‌صورت نظام‌مند از یک چارچوب به چارچوب دیگر منتقل کنند، رابط‌های منسوخ را به‌روز کنند و مجموعه‌تست‌ها را نوسازی کنند؛ کارهایی که پیش‌تر آن‌قدر خسته‌کننده و پرخطر بودند که عملاً هیچ‌وقت انجام نمی‌شدند.

مدل کارخانه: ساختن سامانه‌ای که نرم‌افزار می‌سازد

همهٔ این تغییرها را می‌توان در قالب یک مدل ذهنی کنار هم دید که آن را **مدل کارخانه** (Factory model) می‌نامیم. در این مدل، محصول اصلی کار توسعه‌دهنده دیگر خودِ کد نیست، بلکه **سامانه‌ای است که کد را تولید می‌کند**. این سامانه از این اجزا تشکیل شده است:

- مشخصات و زمینه‌ای که تعریف می‌کند چه باید ساخته شود
- عامل‌هایی که مشخصات را به پیاده‌سازی ترجمه می‌کنند
- تست‌ها و دروازه‌های کیفیت که درستیِ کار را راستی‌آزمایی می‌کنند
- حلقه‌های بازخورد که خطاها را برای اصلاح به عامل‌ها برمی‌گردانند
- گاردریل‌هایی که رفتار عامل‌ها را ایمن و قابل‌پیش‌بینی نگه می‌دارند

مدیر کارخانه قطعه‌ها را خودش با دست سرِهم نمی‌کند؛ خط مونتاژ را طراحی می‌کند و از کیفیت محصول اطمینان می‌یابد. توسعه‌دهندهٔ امروزی هم سامانهٔ توسعه را طراحی می‌کند و مطمئن می‌شود خروجی آن به استاندارد لازم می‌رسد. **راه موفقیت این است که به‌جای دستورهای گام‌به‌گام، معیارهای موفقیت را به عامل‌ها بدهید و بگذارید آن‌قدر تکرار کنند تا به آن برسند.**



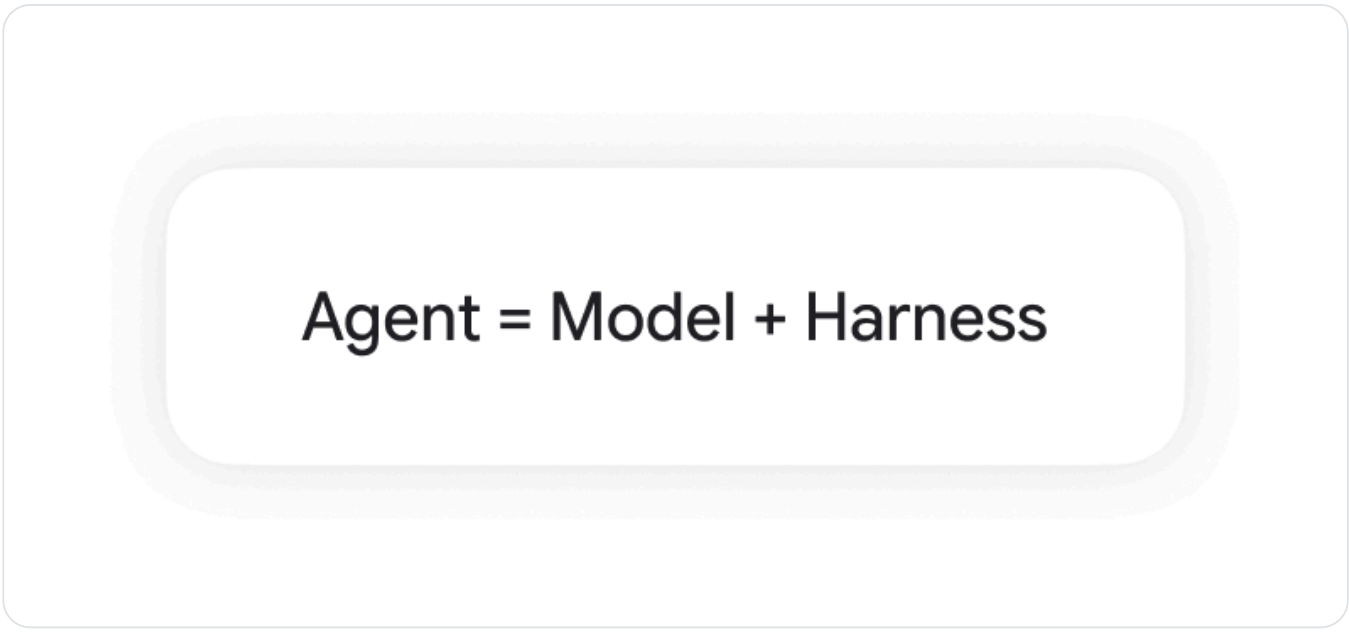
شکل ۶: مدل کارخانه — توسعه‌دهنده سامانه را طراحی می‌کند ← عامل‌ها کد تولید می‌کنند ← تست‌ها خروجی را راستی‌آزمایی می‌کنند.

۴. مهندسی مهار: آنچه دور مدل ساخته می‌شود

اگر توسعه‌دهنده مدیر کارخانه باشد، مدل هوش مصنوعی فقط موتوری خام در کارگاه است. موتور به‌تنهایی نمی‌تواند خودرو بسازد؛ به تسمه، چرخ‌دنده، حسگر ایمنی و خط مونتاژ هم نیاز دارد. در توسعهٔ مبتنی بر هوش مصنوعی، به این تجهیزات پیرامونی **مهار** (harness) می‌گویند.

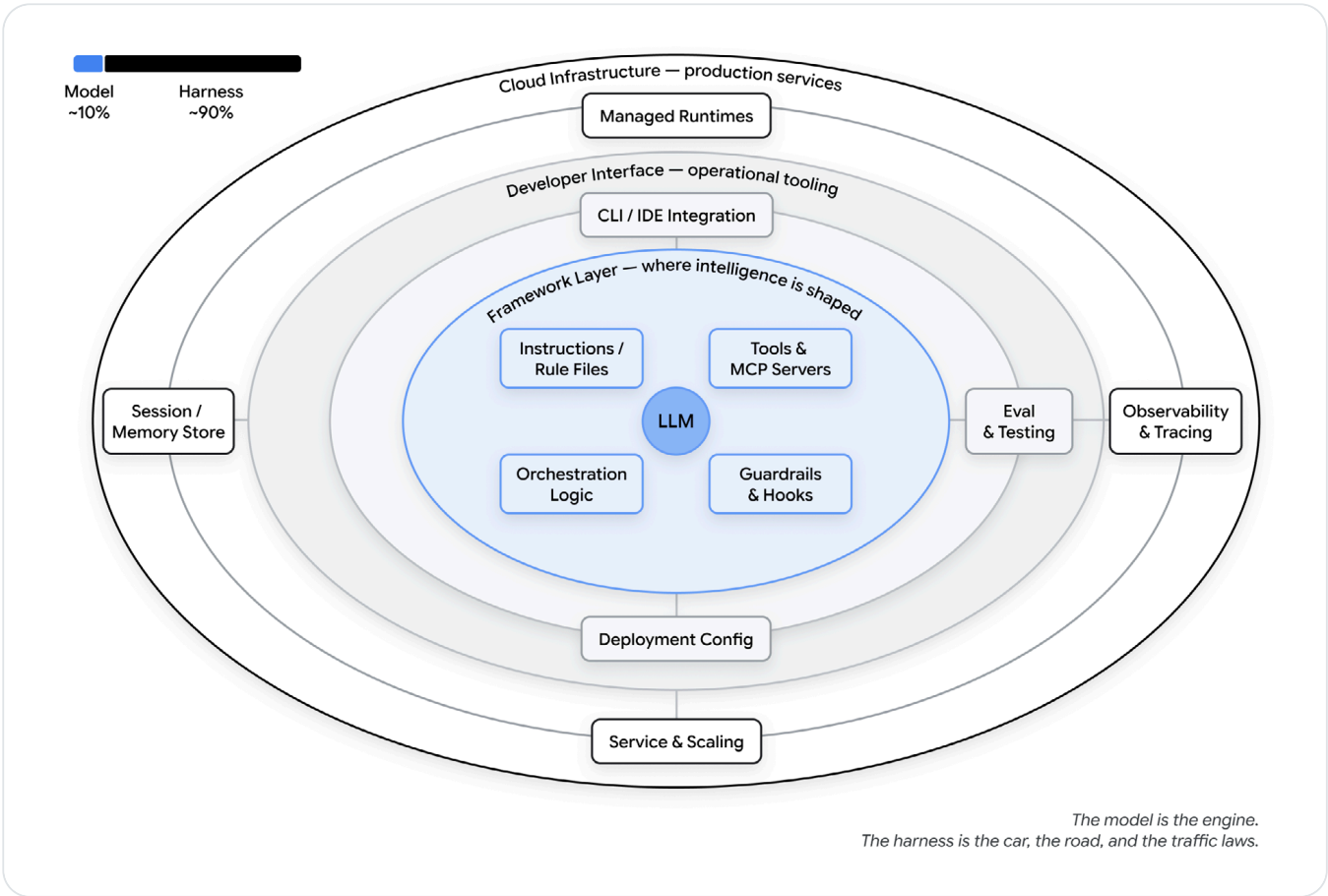
کسانی که تازه ساختن عامل را شروع می‌کنند، وسوسه می‌شوند مدل را همان کل سامانه بدانند: مدل تازه‌ای می‌آید و عامل باهوش‌تر می‌شود؛ مدل قدیمی‌تر باشد، عامل ضعیف‌تر می‌شود. در این نگاه، هر نقطهٔ قوت یا ضعفی به مدل نسبت داده می‌شود.

این برداشت غلط است و به سرمایه‌گذاری‌های اشتباه می‌انجامد. مدل فقط یکی از اجزای عاملی است که در حال کار است. هر چیز دیگری — پرامپت‌ها، ابزارها، سیاست‌های (policy) زمینه، قلاب‌ها، سندباکس‌ها، زیرعامل‌ها، رصدپذیری (observability) — **مهار** است: داربستی که دور مدل ساخته می‌شود و به آن امکان می‌دهد کاری را واقعاً به سرانجام برساند.



شکل ۷: معادلهٔ کلیدی — عامل = مدل + مهار.

مدل خام به‌تنهایی عامل نیست؛ وقتی به عامل تبدیل می‌شود که مهار، وضعیت، امکان اجرای ابزار، حلقهٔ بازخورد و قیدهای الزام‌آور را در اختیارش بگذارد. رفتاری که توسعه‌دهندگان هنگام کار با عامل‌های کدنویسی می‌بینند، بیش از هر چیز به کاری بستگی دارد که **مهار** انجام می‌دهد، نه فقط به اینکه کدام مدل پشت آن است.



شکل ۸: آناتومی مهار.

درون مهار چیست

- **دستورها و فایل‌های قاعده:** متنی که تعریف می‌کند عامل کیست، چه چیزی برایش اهمیت دارد و چه کارهایی برایش ممنوع است؛ از جمله فایل‌های راهنمای عامل، فایل‌های مهارت و پرامپت‌های زیرعامل (subagent).
- **ابزارها:** توابع، سرورهای پروتکل زمینهٔ مدل (Model Context Protocol – MCP) و رابط‌هایی که عامل می‌تواند فراخوانی کند، به‌همراه توضیحاتی که می‌گوید کِی و چگونه از آن‌ها استفاده کند.
- **سندباکس‌ها** (sandbox) **و محیط‌های اجرا:** جایی که کد عامل واقعاً اجرا می‌شود، و اینکه به چه چیزهایی دسترسی دارد و به چه چیزهایی ندارد.
- **منطق ارکستراسیون:** راه‌اندازی زیرعامل‌ها، مسیریابی (routing) میان مدل‌ها، واگذاری کار میان عامل‌های متخصص، و قواعدی که تعیین می‌کنند هرکدام کِی فعال شوند.
- **گاردریل‌ها یا قلاب‌ها** (hook)**:** کد قطعی که در نقاط مشخصی از چرخهٔ عمر اجرا می‌شود: پیش از فراخوان ابزار، پس از ویرایش فایل، پیش از کامیت. قلاب‌ها برای کارهایی‌اند که عامل هرگز نباید فراموش کند، اما اغلب فراموش می‌کند.
- **رصدپذیری:** لاگ‌ها، ردها، ارزیابی‌ها و اندازه‌گیری هزینه و تأخیر (latency). بدون رصدپذیری، راهی نیست که بفهمید عامل خوب کار می‌کند یا بی‌صدا از مسیر خارج می‌شود.

اگر این محدوده گسترده به‌نظر می‌رسد، واقعاً هم گسترده است؛ و **مسئولیت آن با تیم شماست، نه ارائه‌دهندهٔ مدل.**

مهار در چرخهٔ تولید

مدل تعیین می‌کند کار **چگونه** انجام شود، و مهار داربستی است که ابزارها، سندباکس‌ها و ارکستراسیونِ لازم برای اجرای آن را در اختیارش می‌گذارد. به همین دلیل، مهار باید در هر مرحله‌ای که عامل در آن کار می‌کند حضور داشته باشد.

مرحله	اجزای مهار	کنش
۱. نیازمندی، برنامه‌ریزی و معماری پیکربندی مهار	دستورها و فایل‌های قاعده	توسعه‌دهنده فایل‌های قاعده را می‌سازد و قیدهای معماری را تعریف می‌کند، ابزارهایی را که عامل به آن‌ها دسترسی خواهد داشت مشخص می‌کند، و قواعد بنیادینی را که عامل نمی‌تواند بشکند می‌گذارد.
۲. پیاده‌سازی اجرای مهار	سندباکس‌ها، محیط‌های اجرا، ابزارها	همان‌طور که مدل کد تولید می‌کند، آن را درون سندباکس ایزولهٔ مهار اجرا می‌کند. اگر مدل نیاز به خواندن فایل یا جست‌وجوی وب داشته باشد، از ابزارهایی که مهار فراهم کرده استفاده می‌کند.
۳. تست و تضمین کیفیت حلقهٔ بازخورد	منطق ارکستراسیون، گاردریل‌ها	مهار محیط اجرا را فراهم می‌کند تا تست‌های خودکار اجرا شوند. اگر تستی شکست بخورد، منطق ارکستراسیون خروجی خطا را می‌گیرد و به مدل برمی‌گرداند و از او می‌خواهد دوباره تلاش کند. **مهار همان چیزی است که این حلقهٔ «فکر ← کنش ← مشاهده» را می‌سازد.**
۴. بازبینی، دیپلوی و نگه‌داری مشاهدهٔ مهار	قلاب‌ها، رصدپذیری	مهار قلاب‌های قطعی را اجرا می‌کند (مثلاً جلوگیری از کامیت، اگر عامل بخواهد گذرواژه‌ای سخت‌کدشده `(hardcoded)` را بفرستد). لایهٔ رصدپذیری هزینهٔ توکن، تأخیر و انحراف `(drift)` عامل را ردیابی می‌کند و اجازه می‌دهد مهندسان دقیقاً ممیزی کنند چرا عامل تصمیم دیپلوی مشخصی گرفت.

اثر مهار اندازه‌پذیر است

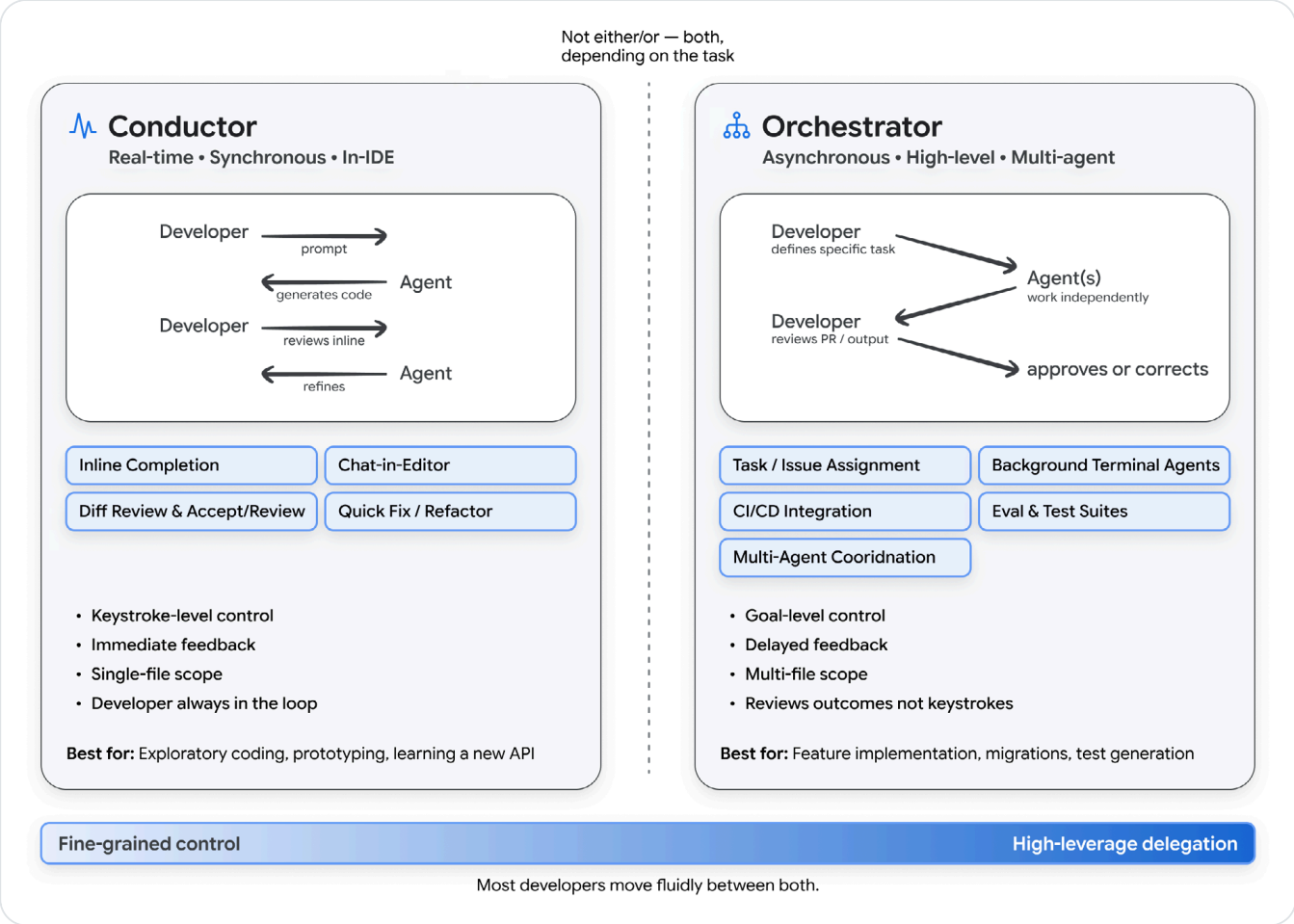
گذار از «کدنویسی شهودی» به «مهندسی عامل‌محور» صرفاً دربارهٔ ابزارهایی که به کار می‌برید نیست — توسعه‌دهنده می‌تواند با **دقیقاً همان عامل** کدنویسی شهودی کند یا مهندسی عامل‌محور. تفاوت در این است که **چقدر آگاهانه مهار را پیکربندی و اعمال می‌کنید.**

بنچمارک‌های (benchmark) عمومی اندازهٔ این اثر را ملموس می‌کنند. روی یکی از بنچمارک‌های شناخته‌شده، تیمی عاملی را با تغییر **فقط مهار** و بدون هیچ تغییر مدلی، از بیرون رتبهٔ ۳۰ به جمع پنج عامل برتر رساند. مطالعه‌ای جداگانه امتیاز عاملی را روی همان بنچمارک، تنها با تنظیم سیستم پرامپت، ابزارها و میان‌افزار پیرامون مدلی ثابت، **۱۳٫۷ واحد** بالا برد.

نسخهٔ روزمرهٔ این مشاهده حیاتی است: وقتی عامل کاری را اشتباه انجام می‌دهد، نخستین غریزه سرزنش مدل است. اما اغلب ریشهٔ خطا در ابزاری است که وجود ندارد، قاعده‌ای است که مبهم تعریف شده، گاردریلی که اصلاً تعریف نشده، یا کانتکست ویندوای پر از نویز است. **بیشتر خطاهای عامل، اگر صادقانه بررسی شوند، در واقع خطای پیکربندی‌اند.**

۵. نقش در حال تحول توسعه‌دهنده

هرچه هوش مصنوعی بخش بیشتری از کار پیاده‌سازی را بر عهده می‌گیرد، نقش توسعه‌دهنده هم بیشتر دگرگون می‌شود؛ تغییری که هم هیجان‌انگیز است و هم گیج‌کننده. برای فهم این تغییر، بهتر است دو حالت کاری را در نظر بگیریم که توسعه‌دهندگان به‌راحتی میان آن‌ها جابه‌جا می‌شوند: **رهبر ارکستر** (Conductor) و **هماهنگ‌کننده** (Orchestrator).



شکل ۹: رهبر ارکستر در برابر هماهنگ‌کننده — دو حالت کار با عامل‌ها.

رهبر ارکستر: هدایت مستقیم و هم‌زمان

در حالت رهبر ارکستر، توسعه‌دهنده هم‌زمان و دوشادوش یک هوش مصنوعی، مثل یک همکار در برنامه‌نویسی جفتی (`pair programming`)، کار می‌کند. ویرایشگر در مقابل توسعه‌دهنده باز است و او نوشته‌شدن کد را می‌بیند، با پرامپت و اصلاح هوش مصنوعی را هدایت می‌کند، و کنترل دقیقی بر آنچه نوشته می‌شود دارد. هوش مصنوعی مثل سازی قدرتمند است، اما رهبری هر حرکت با توسعه‌دهنده است.

این حالت معمولاً وقتی به کار می‌آید که روی منطقی پیچیده کار می‌کنید، مشکلی دشوار را دیباگ می‌کنید، یا در کدبیسی ناآشنا هستید و باید هر تغییر را همان لحظه بفهمید. برای توسعه‌دهندگانی که پیشینهٔ مهندسی سنتی دارند، این حالت طبیعی است و همان حس فهم و کنترلی را حفظ می‌کند که بسیاری از مهندسان برایش ارزش قائل‌اند. **خطرش این است که خود توسعه‌دهنده به گلوگاه تبدیل شود**؛ اگر قرار باشد او تک‌تک ضربه‌های کلید را هدایت کند، سرعت کار چندان بیشتر نمی‌شود.

هماهنگ‌کننده: واگذاری ناهم‌زمان و چندعاملی

در حالت هماهنگ‌کننده، توسعه‌دهنده در سطح انتزاع بالاتری کار می‌کند: هدف‌ها را تعریف می‌کند، آن‌ها را به عامل‌ها می‌سپارد و نتیجه را بازبینی می‌کند، اما دیگر نوشته‌شدن کد را خط‌به‌خط دنبال نمی‌کند. عامل‌ها ممکن است در پس‌زمینه و به‌طور موازی روی بخش‌های مختلف کدبیس کار کنند. توسعه‌دهنده هر از گاهی سر می‌زند، خروجی را بازبینی می‌کند و در صورت لزوم مسیر را اصلاح می‌کند.

این حالت برای کارهایی مناسب است که به‌خوبی تعریف شده‌اند: رفع اشکال، پیاده‌سازی قابلیت بر پایهٔ الگوهای موجود، مهاجرت کدبیس و تولید تست. حالت هماهنگ‌کننده مهارت‌های متفاوتی می‌طلبد؛ به‌جای تسلط عمیق بر نحو و ریزه‌کاری‌های زبان، این مهارت‌ها لازم است:

- **مشخصات‌نویسی:** تعریف کارها آن‌قدر دقیق که عامل بتواند بدون ابهام انجامشان دهد.
- **تجزیه** `(decomposition)`**:** تقسیم کارهای بزرگ به بخش‌هایی که اندازه‌شان برای انجام‌دادن به‌دست عامل مناسب باشد.
- **ارزیابی:** سنجش سریع اینکه خروجی عامل به استانداردهای کیفیت می‌رسد یا نه.
- **طراحی سامانه:** طراحی قیدها، تست‌ها و حلقه‌های بازخوردی که کار عامل‌ها را بهره‌ور نگه می‌دارند.

مسئلهٔ ۸۰ درصد

> **۸۰ درصد آسان، ۲۰ درصد سخت**
>
> یکی از چالش‌های همیشگی توسعهٔ مبتنی بر هوش مصنوعی همان چیزی است که آن را **مسئلهٔ ۸۰ درصد** `(The 80% problem)` می‌نامیم: عامل‌ها می‌توانند حدود ۸۰ درصد از کد یک قابلیت را به‌سرعت تولید کنند، اما ۲۰ درصد باقی‌مانده — یعنی حالت‌های مرزی، مدیریت خطا، نقاط اتصال به بخش‌های دیگر سامانه، و جزئیات حساسی که درستی کار به آن‌ها بستگی دارد — شناخت عمیقی از بستر پروژه می‌خواهد که مدل‌های کنونی اغلب ندارند.

ماهیت خطاهای هوش مصنوعی هم تغییر کرده است: از اشتباه‌های سادهٔ نحوی به خطاهای مفهومیِ پنهان‌تر و خطرناک‌تر رسیده‌ایم؛ مثل پیش‌فرض‌های غلط دربارهٔ منطق کسب‌وکار، نپرسیدن سؤال وقتی نیازمندی‌ها مبهم‌اند، نادیده‌گرفتن حالت‌های مرزی، و تصمیم‌های معماری‌ای که در درازمدت بار نگه‌داری پنهانی می‌سازند. **تشخیص این خطاها دقیقاً به این دلیل سخت‌تر است که کد «درست به‌نظر می‌رسد» و حتی ممکن است تست‌های پایه را هم پاس کند.**

توسعه‌دهندگانی که از پس این چالش بهتر برمی‌آیند، رویکرد روشنی دارند: هوش مصنوعی را برای کارهایی به کار می‌برند که در آن‌ها خوب است (پیاده‌سازی سریع کارهایی که به‌خوبی تعریف شده‌اند) و توجه خودشان را برای جاهایی نگه می‌دارند که هوش مصنوعی در آن‌ها ضعیف است (نیازمندی‌های مبهم، موازنه‌های معماری، و

راستی‌آزمایی درستی کار). آن‌ها سرعت را از پذیرفتن هر چیزی که هوش مصنوعی تولید می‌کند به دست نمی‌آورند؛ از متمرکزکردن تخصصشان روی جایی به دست می‌آورند که بیشترین اهمیت را دارد.

۶. عامل‌های کدنویسی در عمل

جای عامل‌ها در روز کاری توسعه‌دهنده

عامل‌های کدنویسی در سه جا در کار روزمره ظاهر می‌شوند. بیشتر توسعه‌دهندگان هر سه را هم‌زمان به کار می‌برند.

در ویرایشگر

تکمیل درون‌خطی که هم‌زمان با تایپ توسعه‌دهنده، خط بعدی را پیشنهاد می‌کند؛ پنل‌های گفت‌وگو که کد را همان‌جا توضیح می‌دهند یا تغییر می‌دهند؛ و آگاهی از کل کدبیس درون خود ویرایشگر. بیشتر افراد نخستین بار همین‌جا با هوش مصنوعی در کدنویسی آشنا می‌شوند، و توسعه‌دهنده بی‌آنکه از ویرایشگر بیرون برود، تمرکز کارش را حفظ می‌کند.

در ترمینال

توسعه‌دهنده عامل را از خط فرمان راه می‌اندازد، هدف را به زبان ساده به آن می‌گوید و می‌گذارد در سراسر کدبیس کار کند. این عامل‌ها به کل فایل‌سیستم دسترسی دارند، چند فایل را با هم ویرایش می‌کنند، و می‌توانند ابزارها و تست‌ها را اجرا کنند و بر پایهٔ نتیجه، کار را دوباره تکرار کنند. **امروز جدی‌ترین کدنویسی شهودی همین‌جا اتفاق می‌افتد.**

در پس‌زمینه

این عامل‌ها کاری را تحویل می‌گیرند و به‌صورت خودگردان، گاهی چند ساعت، در سندباکس‌های ابری روی آن کار می‌کنند و معمولاً در پایان یک پول‌ریکوئست تحویل می‌دهند. توسعه‌دهنده کار را به آن‌ها می‌سپارد و بعداً نتیجه را بازبینی می‌کند.

در عمل، عامل ویرایشگر وقتی به کار می‌آید که توسعه‌دهنده وسط نوشتن کد است و بی‌آنکه تمرکزش به هم بخورد، پیشنهاد، ویرایش سریع یا توضیح می‌خواهد. عامل ترمینال برای کارهای چندفایلی، کاوش کدبیس‌های ناآشنا، و کارهایی مناسب است که در آن‌ها عامل باید کد را اجرا کند و به نتیجه‌ای که می‌بیند واکنش نشان دهد. عامل پس‌زمینه برای کارهایی مناسب است که به‌خوبی مشخص شده‌اند؛ کارهایی که توسعه‌دهنده می‌تواند در یک پاراگراف توصیفشان کند و سراغ کار دیگری برود. **انتخاب درست به ماهیت کار بستگی دارد، نه به اینکه کدام نوع عامل خودگردان‌تر است.**

کدنویسی شهودیِ عامل‌های آمادهٔ تولید

هرچه تا اینجا گفتیم دربارهٔ **استفاده** از عامل‌های کدنویسی برای ساخت نرم‌افزار بود. اما اگر چیزی که می‌خواهید بسازید خودش یک عامل باشد، چه؟

مثلاً رباتی برای پشتیبانی مشتری که درخواست‌های بازپرداخت را رسیدگی می‌کند، دستیاری پژوهشی که منابع را با هم مقایسه می‌کند و گزارش‌های مستند می‌نویسد، یا ابزاری داخلی که رعایت مقررات را پایش می‌کند و موارد غیرعادی را گزارش می‌دهد. این‌ها کارهایی نیستند که با یک عامل در ترمینال حلشان کنید؛ این‌ها **محصول‌اند** و ابزار، حافظه، ارزیابی و زیرساخت دیپلوی مخصوص خودشان را لازم دارند.

همان گردش‌کار مبتنی بر ترمینال که با آن اسکریپت‌های نمونه ساخته می‌شوند، حالا برای ساختن این عامل‌های تولیدی هم به کار می‌رود. ساخت، ارزیابی و دیپلوی یک عامل واقعی که در مقیاس بزرگ کار می‌کند — با حافظهٔ ماندگار، حکمرانی و رصدپذیری — دیگر کاری نیست که فقط با چارچوب‌ها و کنسول‌های ابری انجام شود؛ حالا در همان ترمینال انجام می‌شود، و اغلب با گفت‌وگو با همان عامل کدنویسی‌ای که توسعه‌دهنده از قبل با آن کار می‌کند.

کِی این گردش‌کار اهمیت دارد

وقتی به عاملی نیاز دارید که بتوان برای کاربران واقعی به آن اتکا کرد؛ یعنی عاملی با حافظهٔ ماندگار میان نشست‌ها، مجوزهای محدود و مشخص روی ابزارها و داده‌ها، ارزیابی‌هایی که پس‌رفت‌ها `(regression)` را پیش از انتشار پیدا می‌کنند، و رصدپذیری‌ای که نشان می‌دهد عامل واقعاً چه کرده است.

برای اسکریپت‌های یک‌بارمصرف یا خودکارسازی کارهای شخصی، یک عامل کدنویسی معمولی کافی است؛ آنجا **عامل فقط ابزار کار است**. اما برای عامل‌هایی که در مقیاس بزرگ به کاربران واقعی خدمت می‌دهند، **عامل خودِ محصول است** و به یک بستر زیرساختی نیاز دارد.

```
# One-time setup
uvx google-agents-cli setup

# Then in your coding agent:
> Build a support agent that answers questions from our docs.
> evaluate it on the FAQ dataset
> Deploy it to Agent Engine
```

قطعهٔ ۱: راه‌اندازی و ساخت با خط فرمان عامل‌ها.

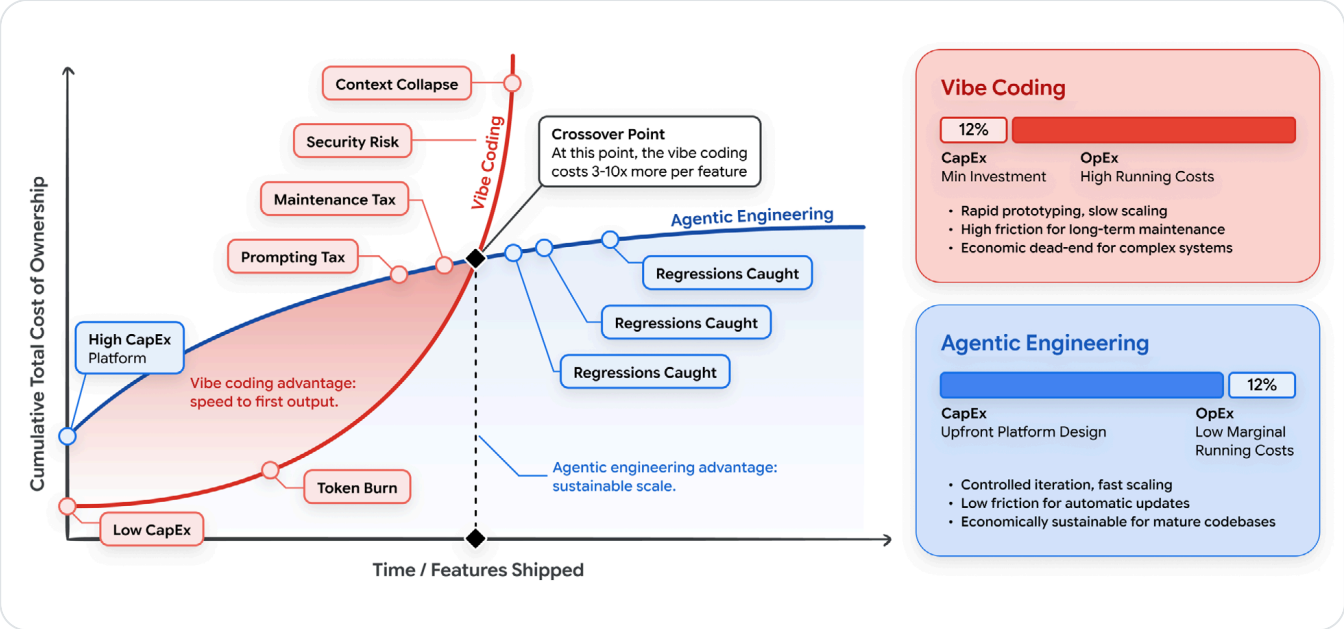
پشت همان یک دستور (instruction)، عامل کدنویسی این کارها را انجام می‌دهد: اسکلت پروژه را از روی یک قالب می‌سازد، کد را می‌نویسد، یک مجموعه‌ارزیابی تولید می‌کند و آن را روی عامل اجرا می‌کند، عامل را روی رانتایم (runtime) مستقر می‌کند و در پایان گزارش می‌دهد. **پیش‌تر، عامل‌های تولیدی به پشتهٔ فناوری و گردش‌کاری جدا از نمونه‌های اولیه نیاز داشتند. حالا نمونهٔ اولیه‌ای که دیروز روی لپ‌تاپ توسعه‌دهنده اجرا می‌شد، امروز می‌تواند بدون بازنویسی به عاملی تولیدی تبدیل شود که به کاربران واقعی خدمت می‌دهد.**

همین گردش‌کار را می‌توان از یک عامل به چند عامل گسترش داد. هماهنگی میان عامل‌ها در حالت‌های ساده از راه وضعیت مشترک نشست انجام می‌شود، دسترسی به ابزار از راه MCP، و واگذاری (handoff) کار میان عامل‌ها از راه پروتکل عامل‌به‌عامل (A2A). در آزمایشی که اوایل ۲۰۲۶ منتشر شد، تیم‌هایی از عامل‌ها که روی چنین معماری‌ای کار می‌کردند، ظرف دو هفته یک کامپایلر کارآمد برای زبان C ساختند؛ در این مدت انسان‌ها جهت کار را تعیین و خروجی را بازبینی می‌کردند، اما خودشان کد پیاده‌سازی را نمی‌نوشتند. **گلوگاه از نوشتن کد به این دو کار منتقل شد: مشخص‌کردن اینکه سامانه چه باید بکند، و راستی‌آزمایی اینکه عامل‌ها واقعاً همان کار را کرده‌اند.**

۷. هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی

وقتی اثر هوش مصنوعی بر چرخهٔ تولید نرم‌افزار را می‌سنجیم، بحث معمولاً فقط حول سرعت توسعه‌دهنده می‌چرخد: چقدر سریع‌تر می‌توانیم کد بنویسیم؟ اما برای مدیران فنی، معیار مهم‌تر **هزینهٔ کل مالکیت** (Total Cost of Ownership) است.

برای فهم هزینهٔ واقعی توسعهٔ مبتنی بر هوش مصنوعی، باید ببینیم هر گردش‌کار، بار مالی و عملیاتی را چگونه میان دو نوع هزینه تقسیم می‌کند: **هزینهٔ سرمایه‌ای** (CapEx)، یعنی سرمایه‌گذاری اولیه برای ساختن چیزی، و **هزینهٔ عملیاتی** (OpEx)، یعنی هزینهٔ جاری اجرا، اصلاح و نگه‌داری. نکتهٔ مهم این است که در دوران هوش مصنوعی، هزینهٔ عملیاتی تا حد زیادی به **اقتصاد توکن** بستگی دارد.



شکل ۱۰: هزینه و صرفهٔ اقتصادی توسعه با هوش مصنوعی.

بدهی پنهان کدنویسی شهودی — سرمایهٔ کم، عملیات گران

در نگاه اول، کدنویسی شهودی فوق‌العاده مقرون‌به‌صرفه به‌نظر می‌رسد. شروع کار تقریباً هیچ هزینه‌ای ندارد: یک اشتراک ماهانه و چند پرامپت سرِدستی. هزینهٔ سرمایه‌ای ناچیز است، چون توسعه‌دهنده به‌جای اینکه وقتش را صرف طراحی سامانه کند، کاملاً به توانایی‌های خودِ مدل تکیه می‌کند. اما اقتصاد کدنویسی شهودی بار عملیاتی عظیمی را پنهان می‌کند که مدام روی هم انباشته می‌شود:

- **هدررفت توکن:** هر تعامل با مدل زبانی هزینه‌ای بر پایهٔ توکن ورودی و خروجی دارد. در کدنویسی شهودی، توسعه‌دهندگان اغلب فایل‌های حجیم و بی‌ساختار را در کانتکست ویندو (`context window`) می‌ریزند و بارها از مدل می‌خواهند اشتباه‌هایی را که خودش ساخته و کسی بررسی‌شان نکرده، درست کند. نتیجه، چرخه‌ای پرهزینه از پرامپت‌دادن‌های پیاپی است که توکن‌ها را هدر می‌دهد، چون کار در همان دفعهٔ اول به‌ندرت درست درمی‌آید.
- **هزینهٔ پنهان نگه‌داری:** کدی که با پرامپت‌های بی‌برنامه نوشته شده، اغلب ساختار منسجمی ندارد. وقتی شش ماه بعد اشکالی پیدا می‌شود، مهندسان باید روزها وقت بگذارند تا از این کد بی‌ساختار سر دربیاورند.
- **هزینهٔ رفع مشکلات امنیتی:** بدون مهاری برای ارزیابی خودکار، تولید سریع کد به تولید سریع آسیب‌پذیری می‌انجامد. هزینهٔ رفع یک نقص امنیتی در محیط تولید، چندین برابر هزینهٔ پیدا کردنش در مرحلهٔ طراحی است.

سرمایه‌گذاری مهندسی عامل‌محور — سرمایهٔ زیاد، عملیات ارزان

مهندسی عامل‌محور این معادلهٔ اقتصادی را برعکس می‌کند. پیش از آنکه حتی یک خط کد تولیدی نوشته شود، باید آگاهانه وقت و منابع مهندسی را سرمایه‌گذاری کرد. این هزینهٔ سرمایه‌ای شامل طراحی اسکیمای `API`ها، ساختن مجموعه‌تست‌های قطعی و، مهم‌تر از همه، سازمان‌دادن به زمینهٔ عامل است. هزینهٔ اولیهٔ این رویکرد بیشتر است،

اما **در عوض، هزینهٔ عرضه و نگه‌داری هر قابلیتِ تازه به‌شدت کاهش می‌یابد.** هوش مصنوعی درون «کارخانه‌ای» کار می‌کند که با قواعد سخت‌گیرانه حکمرانی می‌شود؛ پس خروجی‌اش از نظر ساختاری سالم است، از پیش آزموده شده و با استانداردهای شرکت هم‌راستاست.

مهندسی زمینه به‌مثابه اهرم مالی

در اقتصاد توکن، مهندسی زمینه فقط یک مهارت فنی نیست — یک **راهبرد مالی** است. مدل‌های زبانی برای هر تکه اطلاعاتی که برایشان می‌فرستید هزینه می‌گیرند. فرستادن یک ریپو `(repository)` صدهزارتوکنی در هر پرامپت، وقتی تعداد پرامپت‌ها زیاد شود، از نظر مالی شدنی نیست.

مهندسی زمینهٔ خوب تضمین می‌کند که مدل اطلاعاتی **فشرده و پرسیگنال** دریافت کند، نه اطلاعاتی پراکنده و پر از نویز. وقتی توسعه‌دهندگان از همان ابتدا زمینهٔ درست را فراهم کنند، احتمال اینکه عامل کار را در همان دفعهٔ اول درست انجام دهد به‌شدت بالا می‌رود، و از چرخه‌های پرهزینهٔ آزمون‌وخطا که گریبان کدنویسی شهودی را گرفته، در امان می‌مانند.

افزون بر این، مهندسی عامل‌محور **مسیریابی هوشمند مدل** را ممکن می‌کند. در کدنویسی شهودی، توسعه‌دهنده معمولاً برای هر کاری سراغ یکی از بزرگ‌ترین و پیشرفته‌ترین مدل‌ها می‌رود و گران‌ترین نرخ توکن را می‌پردازد، حتی وقتی فقط می‌خواهد یک غلط تایپی درست شود. مدل کارخانه‌ای که خوب طراحی شده باشد جلوی این هدررفت را می‌گیرد: کارهای بسیار پیچیده را به مدل‌های بزرگ و پیشرفته می‌سپارد، اما کارهای قطعی و ساده را خودکار به مدل‌های کوچک‌تر، سریع‌تر و بسیار ارزان‌تر می‌فرستد.

۸. از کجا شروع کنیم

گذار از نحو به قصد چیزی مربوط به آینده نیست؛ همین امروز پیش روی ماست. چه این را به‌عنوان توسعه‌دهنده‌ای که تنها کار می‌کند بخوانید و چه به‌عنوان مدیری که به آوردن این ابزارها به تیم یا سازمانش فکر می‌کند، یک اصل بنیادی برقرار است: **هوش مصنوعی هر فرهنگ مهندسی‌ای را که در آن به کار گرفته شود، تقویت می‌کند.**

برای تک‌تک توسعه‌دهندگان

1. **یک فایل قواعد برای پروژه بسازید.** قالبی را انتخاب کنید که عامل کدنویسی‌تان آن را می‌شناسد. با ده خط شروع کنید: پشتهٔ فناوری، قراردادهای کدنویسی، قواعد سخت و گردش‌کار. هر بار که عامل کاری کرد که نباید تکرار شود، یک قاعده به فایل اضافه کنید.
2. **مجموعه‌ای از مهارت‌ها نصب کنید** که به شما کمک کنند عامل‌ها را بسازید، ارزیابی کنید، مستقر کنید و بهینه کنید.
3. **یک کار تکراری را انتخاب کنید و آن را به نخستین عاملتان تبدیل کنید.** ساختن یک عامل از ابتدا تا انتها، بیشتر از خواندن دربارهٔ صد عامل به شما یاد می‌دهد.

4. **تست‌ها و ارزیابی‌ها را پیش از تولید کد بنویسید.** تست‌ها و ارزیابی‌ها در کنار هم، قرارداد شما با هوش مصنوعی‌اند. مجموعه‌ای که خوب نوشته شده باشد، قصد شما را دقیق‌تر از هر پرامپتی به زبان طبیعی منتقل می‌کند و توسعهٔ مبتنی بر هوش مصنوعی را از کدنویسی شهودی به مهندسی عامل‌محور می‌رساند.
5. **هر خطی را که قرار است منتشر شود بازبینی کنید.** به هر کدی که بیش از حد زیرکانه به‌نظر می‌رسد شک کنید. بررسی کنید که بسته‌هایی که کد وارد (import) می‌کند واقعاً وجود داشته باشند. کدی که تیم آن را نمی‌فهمد، در آینده هزینهٔ دیباگی به بار می‌آورد که تیم از پسش برنمی‌آید.
6. **مهارت‌های برنامه‌نویسی‌تان را حفظ کنید.** هوش مصنوعی کارهای روزمره را انجام می‌دهد تا توسعه‌دهنده بتواند روی کارهای چالش‌برانگیز تمرکز کند. اما این تقسیم کار فقط وقتی جواب می‌دهد که مهارت‌های بنیادی — دیباگ، طراحی سامانه، و درک شهودی از کارایی و درستی — همچنان قوی بمانند.

برای مدیران فنی

1. **مهندسی زمینه را به یکی از رویه‌های اصلی تیم تبدیل کنید.** با فایل‌های قاعده، سیستم پرامپت‌ها، مجموعه‌های ارزیابی و کتابخانه‌های مهارت مثل کد رفتار کنید: در پول‌ریکوئست بازبینی شوند، همراه پروژه نسخه‌بندی شوند و مالک مشخصی داشته باشند.
2. **معیار پذیرش را ارزیابی بگذارید، نه نمایش.** یک نمایش موفق فقط ثابت می‌کند عامل می‌تواند یک بار موفق شود؛ مجموعهٔ ارزیابی‌ای که پاس شده ثابت می‌کند عامل به‌طور پیوسته موفق می‌شود. اما ارزیابی‌ای که معیار روشنی نداشته باشد، در واقع چیزی را نمی‌سنجد.
3. **فرایند بازبینی کد را متناسب با کدی که هوش مصنوعی تولید می‌کند از نو طراحی کنید،** و به‌ویژه به این موارد توجه کنید: وابستگی‌های (dependency) توهمی، یعنی بسته‌هایی که اصلاً وجود ندارند؛ مدیریت ناکافی خطا؛ و ایرادهای پنهان در درستی کار که در نگاه اول درست به‌نظر می‌رسند.
4. **در قواعد تیم، مرز کار نمونه‌سازی و کار تولیدی را روشن کنید.** کدنویسی شهودی سرعت مناسب برای کاوش را می‌دهد و مهندسی عامل‌محور انضباط لازم برای تولید را. تیم‌هایی که این مرز را مبهم می‌گذارند، در نهایت نمونه‌های اولیه‌ای را ناخواسته منتشر می‌کنند.
5. **روی اجزای مهار به‌عنوان دارایی مشترک تیم سرمایه‌گذاری کنید.** تیم‌هایی که در درازمدت بیشترین سود را از توسعهٔ مبتنی بر هوش مصنوعی می‌برند، همان‌هایی‌اند که مهارشان را یک بار می‌سازند و بارها بهترش می‌کنند.

برای سازمان‌ها

1. **توسعهٔ مبتنی بر هوش مصنوعی را یک سرمایه‌گذاری مهندسی ببینید، نه ابزاری برای افزایش بهره‌وری.** اگر عامل کدنویسی را بدون زیرساخت ارزیابی و رصدپذیری به کار بگیرید، سرعت به دست می‌آورید اما کیفیت را از دست می‌دهید؛ و بدهی فنی‌ای می‌سازید که سریع‌تر از آنچه هر تیمی بتواند تسویه‌اش کند، بزرگ‌تر می‌شود.
2. **پیش از گسترش استفاده، روی زیرساخت تولید سرمایه‌گذاری کنید.** نمونهٔ اولیه‌ای که با کدنویسی شهودی روی لپ‌تاپ ساخته شده، سامانه‌ای تولیدی نیست. چیزی که اولی را به دومی تبدیل می‌کند، انضباط عملیاتی‌ای

است که دورش ساخته می‌شود.

3. **برای ابزارها و ارتباط میان عامل‌ها، استانداردهای باز را انتخاب کنید.** این انتخاب، امروز دستتان را برای ترکیب ارائه‌دهندگان و چارچوب‌های مختلف باز می‌گذارد و فردا شما را از دردسر تغییر سکو نجات می‌دهد.
4. **برای تیم‌هایی برنامه‌ریزی کنید که انسان و عامل را کنار هم دارند،** نه گردش‌کارهایی که فقط انسانی یا فقط عاملی‌اند. فرایند بازبینی کد، نوبت‌کاری و ساختار تیم همه باید تغییر کنند تا این واقعیت را نشان دهند که عامل‌ها حالا عضوی از تیم‌اند، نه صرفاً ابزار.
5. **در استخدام و آموزش، توان قضاوت را محور قرار دهید، نه فقط مهارت پیاده‌سازی را.** ارزشمندترین مهندسان چند سال آینده کسانی خواهند بود که عامل‌ها را خوب هدایت می‌کنند، نه کسانی که بیشترین کد را می‌نویسند.

۹. نتیجه‌گیری: قصد به‌مثابه رابط تازه

گذار از نحو به قصد، پیش‌بینی آینده نیست؛ واقعیت امروز است. توسعه‌دهندگان همین حالا هم بیشتر وقتشان را صرف توصیف چیزی می‌کنند که می‌خواهند، تا اینکه مشخص کنند چگونه باید ساخته شود. چرخهٔ تولید نرم‌افزار هم همین حالا دارد کوتاه‌تر می‌شود، ساختار تازه‌ای پیدا می‌کند و بر پایهٔ توانایی‌های هوش مصنوعی از نو طراحی می‌شود. پرسش این نیست که آیا این دگرگونی رخ می‌دهد یا نه؛ پرسش این است که افراد، تیم‌ها و سازمان‌ها تا چه اندازه موفق از آن عبور خواهند کرد.

سه اصل ماندگار برجسته می‌شوند:

1. **در مقیاس بزرگ، ساختار جواب می‌دهد، نه حس‌وحال.** کدنویسی شهودی برای کاوش، نمونه‌سازی و پروژه‌های شخصی رویکرد درستی است. اما برای نرم‌افزاری که سازمان‌ها به آن تکیه می‌کنند، انضباط مهندسی عامل‌محور یک انتخاب نیست، یک ضرورت است. قطعی‌های سرویس، آسیب‌پذیری‌های امنیتی و دردسرهای بزرگ نگه‌داری درست در فاصلهٔ میان «به‌نظر می‌رسد کار می‌کند» و «در همهٔ شرایط درست کار می‌کند» پیدا می‌شوند.
2. **هوش مصنوعی فرهنگ مهندسی شما را تقویت می‌کند.** سازمان‌هایی که رویه‌های تست قوی، استانداردهای معماری روشن و فرایند بازبینی کد سالمی دارند، سود بسیار بیشتری از توسعهٔ مبتنی بر هوش مصنوعی می‌برند. هوش مصنوعی مثل یک اهرم عمل می‌کند و هم نقاط قوت و هم نقاط ضعف شما را چند برابر می‌کند.
3. **نقش انسان کوچک نمی‌شود، بلکه تغییر می‌کند.** توسعه‌دهندگانی که معماری را می‌فهمند، می‌توانند مشخصات دقیق بنویسند، خروجی را نقادانه ارزیابی کنند و سامانه‌های مؤثری از قیدها و حلقه‌های بازخورد طراحی کنند، امروز بیش از هر زمان دیگری ارزشمندند. مهارت‌های مهم دیگر به پیاده‌سازی محدود نیستند و بیشتر به قضاوت گره خورده‌اند.

ما در آغاز دگرگونی‌ای هستیم که نه‌فقط شیوهٔ ساختن نرم‌افزار، بلکه **اینکه اصلاً چه نرم‌افزاری را می‌توان ساخت** از نو تعریف خواهد کرد. تیم‌های کوچک‌تر از پس مسائل بزرگ‌تری برخواهند آمد. حتی یک توسعه‌دهندهٔ تنها خواهد توانست سامانه‌هایی بسازد و نگه دارد که پیش‌تر به یک دپارتمان کامل نیاز داشتند. و ساختن نرم‌افزار روزبه‌روز آسان‌تر و در دسترس‌تر خواهد شد.

> تولید حل شده است. راستی‌آزمایی، قضاوت و جهت‌دهی، پیشهٔ تازه‌اند.

پی‌نوشت‌ها

مقالهٔ اصلی سی‌ودو پی‌نوشت دارد که به منابع برخط ارجاع می‌دهند. مهم‌ترین منابعی که در متن بالا به آن‌ها استناد شده است:

1. آمار پذیرش دستیارهای کدنویسی هوش مصنوعی، ۲۰۲۵–۲۰۲۶.
2. کارپتی، آ.، «کدنویسی شهودی»، فوریهٔ ۲۰۲۵؛ و «از کدنویسی شهودی تا مهندسی عامل‌محور»، ۲۰۲۶.
3. عثمانی، آ.، «مهندسی عامل‌محور»، «مدل کارخانه»، «مسئلهٔ ۸۰ درصد در کدنویسی عامل‌محور»، و «از رهبران ارکستر تا هماهنگ‌کنندگان».
4. METR، «به‌روزرسانی ارتقا: سنجش اثر ابزارهای کدنویسی هوش مصنوعی»، فوریهٔ ۲۰۲۶.
5. «مقدمه‌ای بر عامل‌ها»، مجموعه‌مقالات عامل‌ها، نوامبر ۲۰۲۵.
6. گزارش‌های صنعتی دربارهٔ بهره‌وری مهندسی نرم‌افزار در عصر هوش مصنوعی، ۲۰۲۵–۲۰۲۶.

دربارهٔ این ترجمه: این متن، ترجمهٔ فارسی مقالهٔ `The New SDLC With Vibe Coding` از مجموعه‌مقالات عامل‌ها (می ۲۰۲۶) است، نوشتهٔ آدی عثمانی، شوبهام سابو و سوکراتیس کارتاکیس. شکل‌ها از سند اصلی برداشته شده‌اند و متن قطعه‌کدها به‌عمد به زبان انگلیسی باقی مانده است، چون دستور اجرایی‌اند. فهرست کامل ارجاع‌ها در سند اصلی موجود است.

واژه‌نامهٔ این جلسه

اصطلاحاتی که در این سند به کار رفته‌اند، به‌همراه معادل انگلیسی‌شان. فهرست کامل 159 اصطلاح دوره در واژه‌نامهٔ دوره آمده است.

اصطلاح	معادل انگلیسی	توضیح
ابزار	`tool`	تابعی که عامل می‌تواند فرا بخواند تا در جهان بیرون کنش کند.
ارزیابی	`evaluation`	سنجش رفتار غیرقطعی با نمره و باند تحمل، در برابر ادعای صفر و یکی تست.
ارکستراسیون	`orchestration`	هماهنگ‌کردن چند عامل یا چند گام اجرا.
افشای تدریجی	`Progressive disclosure`	بارگذاری سه‌سطحی: فراداده همیشه، بدنه هنگام فعال‌شدن، منابع همراه فقط در صورت نیاز.
انحراف	`drift`	دورشدن تدریجی رفتار از آنچه انتظار می‌رفت.
بازیابی	`retrieval`	آوردن دانش بیرونی به زمینه در لحظهٔ نیاز.
بنچمارک	`benchmark`	مجموعه‌کار استاندارد برای مقایسهٔ عملکرد.
تأخیر	`latency`	زمان سپری‌شده تا دریافت پاسخ.
تجزیه	`decomposition`	شکستن کاری بزرگ به زیرکارهای اتمی.
تست	`test`	وارسی قطعی رفتار کد. «یونیت تست» = آزمون واحد.
توکن	`token`	واحد کوچکی از متن که مدل پردازش می‌کند.
حافظه	`memory`	آنچه عامل میان نوبت‌ها یا نشست‌ها نگه می‌دارد.
حافظهٔ رویه‌ای	`Procedural memory`	به‌خاطرسپردن «چگونه انجام دادن»؛ در برابر حافظهٔ رویدادی (چه گذشت) و معنایی (چه می‌دانیم).
حلقه	`loop`	چرخهٔ ادراک ← برنامه ← کنش ← مشاهده که عامل در آن می‌چرخد.
خودگردان	`autonomous`	توان اجرای کار بدون تأیید انسانی در هر گام.

اصطلاح	معادل انگلیسی	توضیح
داور	`judge`	مدلی که خروجی مدل دیگر را در برابر سنجه نمره می‌دهد.
دروازه	`gate`	نقطهٔ وارسی که اجرا را تا برآورده‌شدن شرطی متوقف می‌کند.
دستور	`instruction`	قاعده‌ای که رفتار عامل را شکل می‌دهد، در برابر داده‌ای که پردازش می‌کند.
دیباگ	`debug`	یافتن و رفع علت ریشه‌ای خطا.
دیپلوی	`deploy`	انتشار کد روی محیط اجرا.
رانتایم	`runtime`	محیطی که عامل یا برنامه در آن اجرا می‌شود.
رصدپذیری	`observability`	توان دیدن آنچه درون سامانه رخ می‌دهد.
رهبر ارکستر	`Conductor`	کار هم‌زمان و دستی با عامل در ویرایشگر؛ کنترل دقیق بر هر تغییر.
ریپو	`repository`	مخزن کد تحت کنترل نسخه.
زمینه	`context`	اطلاعاتی که عامل در لحظه در اختیار دارد. پایهٔ «مهندسی زمینه» و «پوسیدگی زمینه».
زمینهٔ ایستا / پویا	`Static`	ایستا همیشه بارگذاری می‌شود (هویت عامل)؛ پویا فقط هنگام نیاز فراخوانی می‌شود (مهارت، نتیجهٔ ابزار، بازیابی سند).
زیرعامل	`subagent`	عاملی که عامل دیگری برای کاری مشخص فرا می‌خواند.
سنجه	`rubric`	معیاری که بر پایه‌اش به خروجی نمره داده می‌شود.
سندباکس	`sandbox`	محیط اجرای ایزوله و کم‌امتیاز برای مهار شعاع انفجاری.

اصطلاح	معادل انگلیسی	توضیح
سیاست	`policy`	قاعدهٔ حکمرانی که بیرون از مدل تعریف و اعمال می‌شود.
عامل	`agent`	مدل به‌علاوهٔ مهار؛ توان استدلال، به‌کارگیری ابزار و اجرای کار.
عامل‌به‌عامل	`A2A`	پروتکل ارتباط میان عامل‌های مستقل؛ زبان مشترکی که از چارچوب، زبان برنامه‌نویسی و ساختار بار بی‌نیاز می‌کند.
فراخوان ابزار	`tool call`	یک نوبت مشخص از به‌کارگیری ابزار، با آرگومان‌ها و نتیجه‌اش.
قصد / نیت	`intent`	خواستهٔ واقعی کاربر، در برابر آنچه لفظاً گفته. ریشهٔ «انحراف قصد» و «رضایت قصد».
قطعی / غیرقطعی	`deterministic`	قطعی یعنی ورودی یکسان همیشه خروجی یکسان می‌دهد؛ مدل‌ها غیرقطعی‌اند.
قلاب	`hook`	کد قطعی که در نقطهٔ مشخصی از چرخهٔ عمر اجرا می‌شود.
قید	`constraint`	محدودیت سختی که فضای کنش عامل را کوچک می‌کند.
مدل کارخانه	`Factory model`	محصول اصلی کار توسعه‌دهنده دیگر خودِ کد نیست، بلکه سامانه‌ای است که کد را تولید می‌کند.
مرز	`boundary`	خط جدایی میان دو ناحیهٔ اعتماد یا مسئولیت.
مسئلهٔ ۸۰ درصد	`The 80% problem`	عامل ۸۰٪ کد را سریع می‌سازد؛ ۲۰٪ باقی‌مانده (حالت‌های مرزی، اتصال به بخش‌های دیگر، جزئیات حساس درستی) شناخت عمیقی از بستر پروژه می‌خواهد.

اصطلاح	معادل انگلیسی	توضیح
مسیر	`trajectory`	توالی گام‌های استدلال و فراخوان ابزار که عامل تا رسیدن به پاسخ طی می‌کند.
مسیریابی	`routing`	انتخاب اینکه کدام مهارت یا عامل کار را بردارد.
مشخصات	`spec`	سند قصد و رفتار مورد انتظار؛ منبع حقیقت برای عامل و انسان.
مهار	`harness`	هرچه پیرامون مدل قرار می‌گیرد: پرامپت، ابزار، سندباکس، ارکستراسیون، قلاب. عامل = مدل + مهار.
مهارت	`skill`	پوشهٔ دانش رویه‌ای که عامل فقط هنگام نیاز بارگذاری می‌کند.
مهندسی زمینه	`Context engineering`	طراحی آگاهانهٔ اطلاعاتی که عامل در اختیار دارد: دستورها، دانش، حافظه، مثال‌ها، ابزارها و گاردریل‌ها.
مهندسی عامل‌محور	`Agentic engineering`	همان قدرت مولد، اما درون سامانه‌ای از مشخصات، تست‌ها، ارزیابی‌ها و گاردریل‌ها با نظارت انسانی بر معماری.
نشست	`session`	کل یک گفت‌وگوی چندنوبتی، از آغاز تا پایان کار.
هماهنگ‌کننده	`Orchestrator`	واگذاری ناهم‌زمان کار به چند عامل و بازبینی نتیجه؛ کار در سطح انتزاع بالاتر.
وابستگی	`dependency`	کتابخانهٔ بیرونی که پروژه به آن تکیه دارد.
واگذاری	`handoff`	سپردن کار از عاملی به عامل دیگر.
پرامپت	`prompt`	متنی که به مدل داده می‌شود. پیش‌تر «درخواست» بود که با request اشتباه می‌شد.

اصطلاح	معادل انگلیسی	توضیح
پس‌رفت	`regression`	خراب‌شدن چیزی که پیش‌تر کار می‌کرد.
پوسیدگی زمینه	`Context rot`	افت کیفیت مدل با بزرگ‌شدن ورودی، حتی وقتی دشواری کار ثابت است.
پوشش	`coverage`	سهمی از رفتار که آزمون یا ارزیابی واقعاً می‌سنجد.
پول‌ریکوئست	`pull request`	درخواست ادغام تغییرها در شاخهٔ اصلی.
کانتکست ویندو	`context window`	بیشینهٔ توکنی که مدل در یک نوبت می‌تواند ببیند.
کد دورریختنی	`Disposable code`	وقتی مشخصات محکم باشد، کل کدبیس بارها بازتولید می‌شود؛ پس دلبستگی عاطفی به کد معنا ندارد.
کدنویسی شهودی	`Vibe coding`	توصیف خواسته به زبان طبیعی، پذیرفتن خروجی مدل و بازگرداندن پیام خطا به مدل تا وقتی کار کند؛ بدون راستی‌آزمایی نظام‌مند.
گاردریل	`guardrail`	قید بیرونی که کنش‌های مجاز عامل را محدود می‌کند.
گردش‌کار	`workflow`	توالی گام‌های یک کار تکرارشونده.
